

第五章：分治法

算法设计与分析

主讲教师：胡一涛

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想
-  设计方法与实例
  - 金块问题
  - 矩阵相乘
  - 大整数乘法
  - 二分搜索

 **考察重点**  **熟练掌握技巧**

□ 分治法的应用实例

-  棋盘覆盖
- 循环赛日程表
-  归并排序
-  快速排序
-  寻找第k小
- 平面点对
- 平面点集凸包问题

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想
- 设计方法与实例
  - 金块问题
  - 矩阵相乘
  - 大整数乘法
  - 二分搜索

□ 分治法的应用实例

- 棋盘覆盖
- 循环赛日程表
- 归并排序
- 快速排序
- 寻找第k小
- 平面点对
- 平面点集凸包问题

考察重点　　熟练掌握技巧

14.1 算法思想（伪币问题）

参见课本：P434-P435

- **例14-1 找出伪币：**给你一个装有16个硬币的袋子。16个硬币中有一个是伪造的，并且那个**伪造的硬币比真的硬币要轻一些**。你的任务是找出这个伪造的硬币。为了帮助你完成这一任务，将提供一台可用来比较两组硬币重量的仪器，利用这台仪器，可以知道两组硬币的重量是否相同。



两两比较，最坏情形需 8 次比较找出伪币

14.1 算法思想（伪币问题）

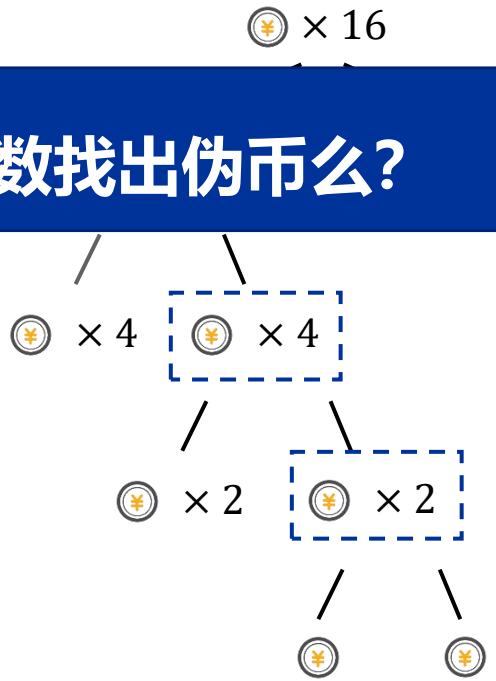
参见课本：P434-P435

- **例14-1 找出伪币：**给你一个装有16个硬币的袋子。16个硬币中有一个是伪造的，并且那个**伪造的硬币比真的硬币要轻一些**。你的任务是找出这个伪造的硬币。为了帮助你完成这一任务，将提供一台可用来比较两组硬币重量的仪器，利用这台仪器，可以知道两组硬币的重量是否相同。

Q：还可以用更少的次数找出伪币么？

每组8枚，找出轻的那组；

- 重复上述过程，找出较轻的4枚、2枚、1枚；
- 仅需 **4 次**比较找出伪币。



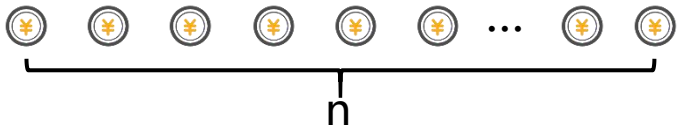
伪币寻找：二等分

1. 假设函数f(n)可以实现所需功能
2. 可以将待解决的大问题分解成小问题
3. 列出边界条件

```
f(A,n)
{
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
                f(A2);
}
```

伪币寻找：二等分

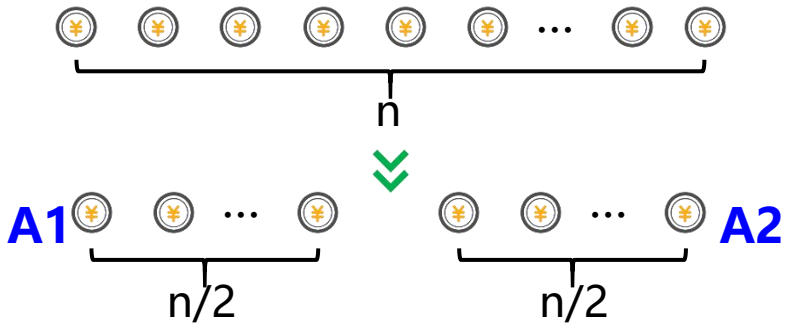
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
                f(A2);
}
```

伪币寻找：二等分

① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个

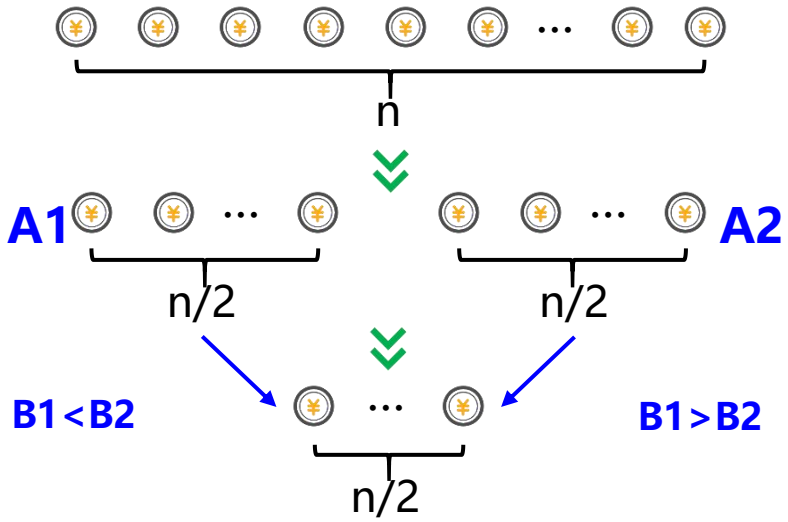



```
f(A,n)
{
    分
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
                f(A2);
}
```

分：如何把大问题分解成小问题?

伪币寻找：二等分

① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个

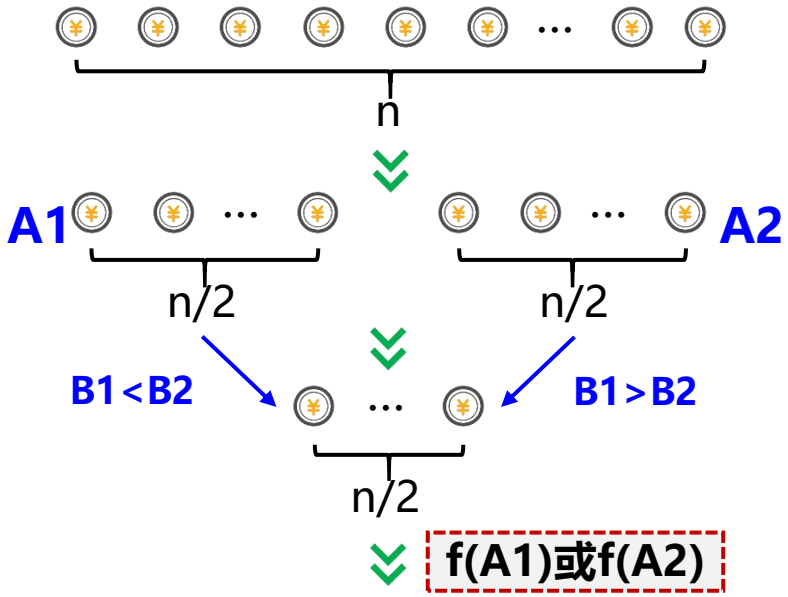



```
f(A,n)
{
    分
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
    治  
                f(A2);
}
```

治：如何求解子问题?

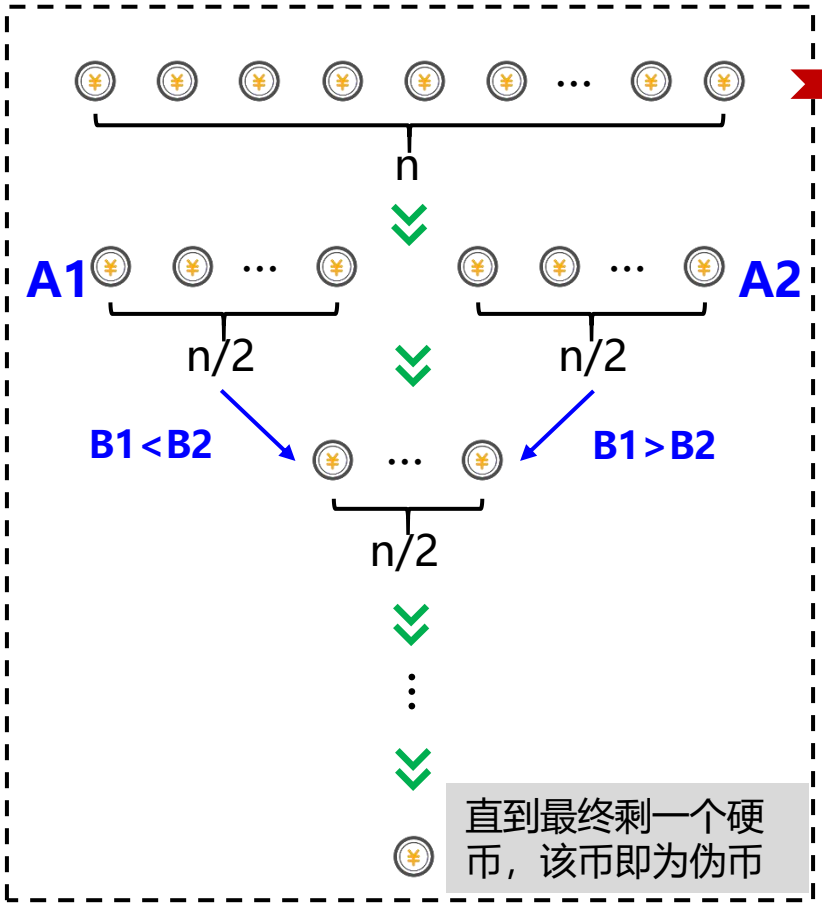
伪币寻找：二等分

① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
                f(A2);
}
```

伪币寻找：二等分边界条件




```
f(A,n)
{
        memcpy(A1,A,n/2);//将A的前n/2个数组放到一起
        memcpy(A2,A+n/2,n/2);//将A的后n/2个数组放到一起
        B1=sum(A1,n/2);
        B2=sum(A2,n/2);
        if(B1<B2) //sum为定义的将数组内的元素加在一起
                f(A1);
        else
                f(A2);
}
```

分解子问题的时间

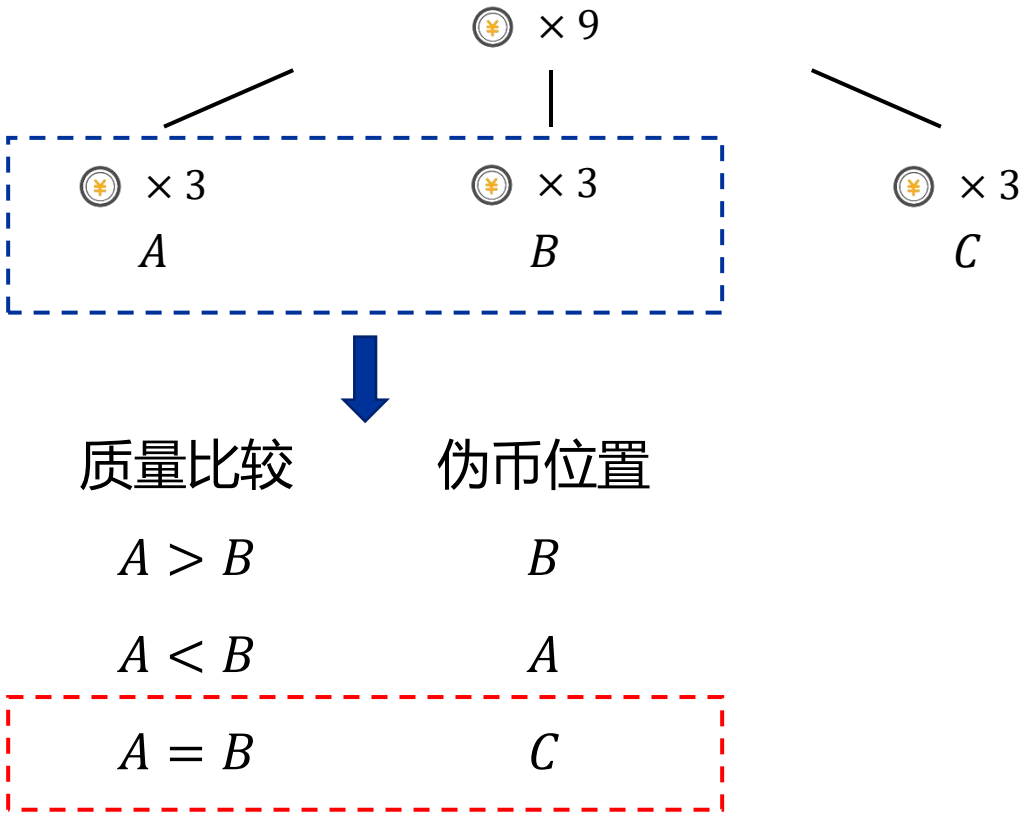
处理子问题的时间

边界条件f(A,n)=1

14.1 算法思想（伪币问题）

参见课本：P434-P435

- 以9个硬币为例：

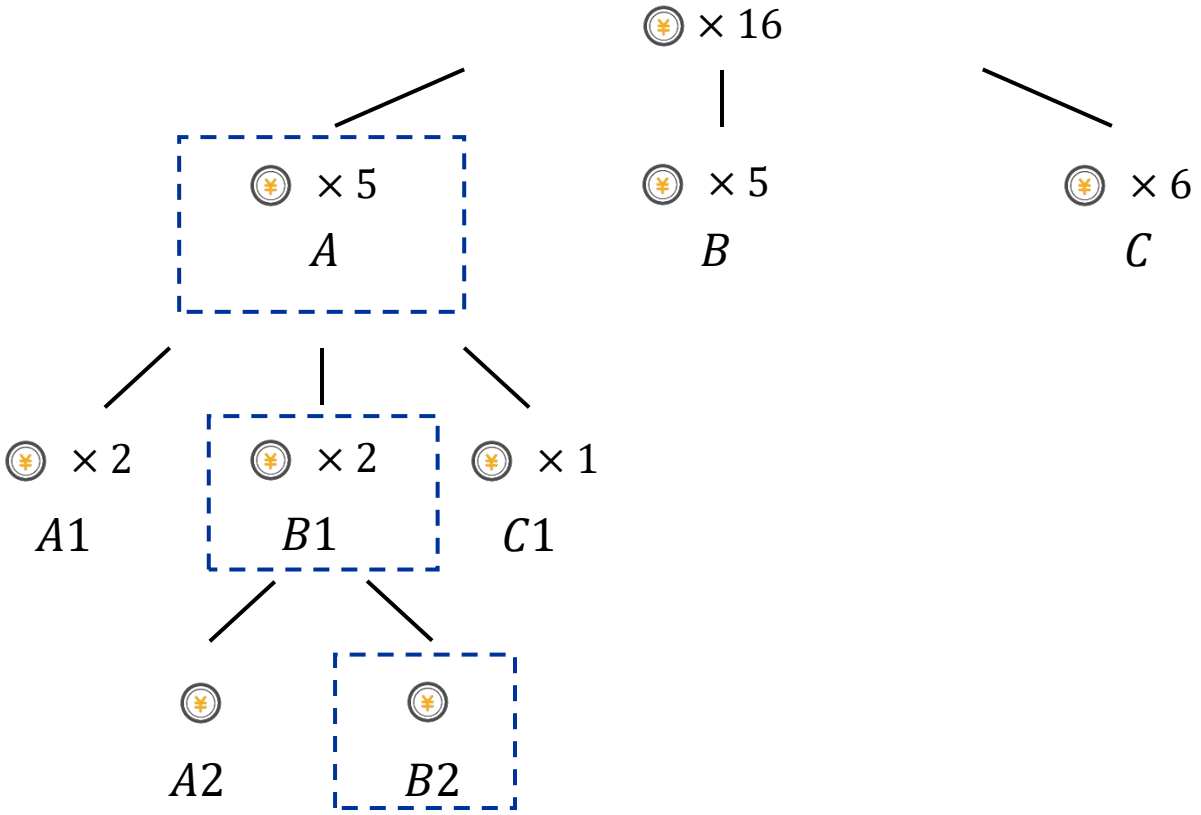


考虑天平外的信息，变为三次拆分的问题

14.1 算法思想（伪币问题）

参见课本：P434-P435

- 合理分解子问题可以提高效率



需要比较次数为：$\lceil log_3$ (硬币总数)$\rceil$

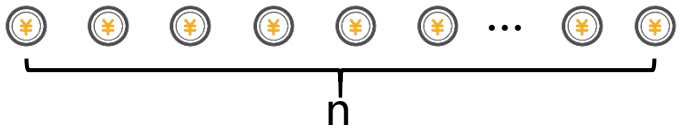
伪币寻找：三等分

1. 假设函数f(n)可以实现所需功能
2. 可以将待解决的大问题分解成小问题
3. 列出边界条件

```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

伪币寻找：三等分

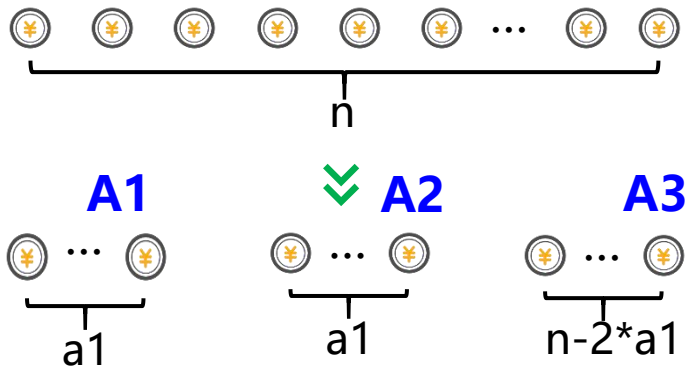
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

伪币寻找：三等分

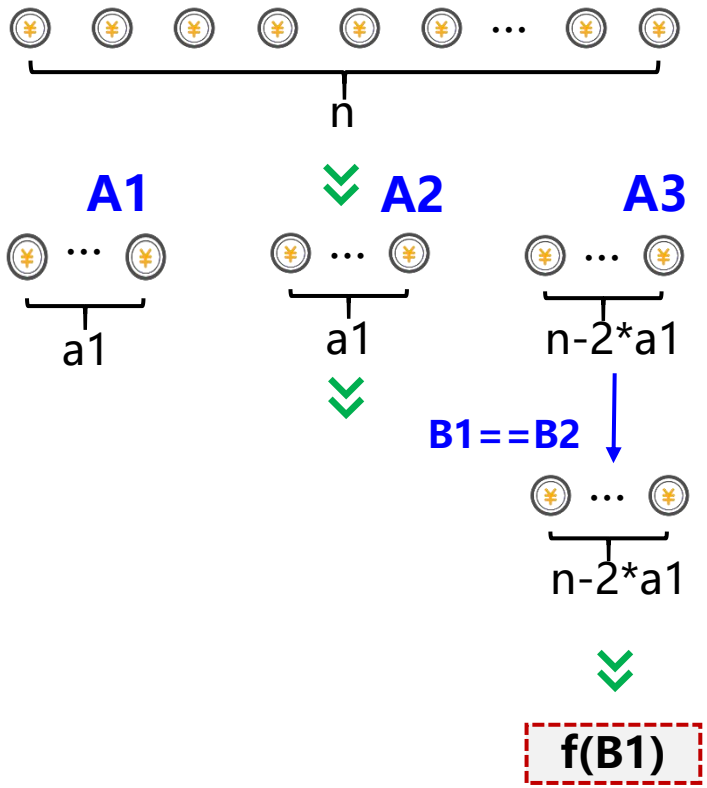
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
    分
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
            f(A3);
        elseif(B1<B2)
            f(A1);
        else
            f(A2);
}
```

伪币寻找：三等分

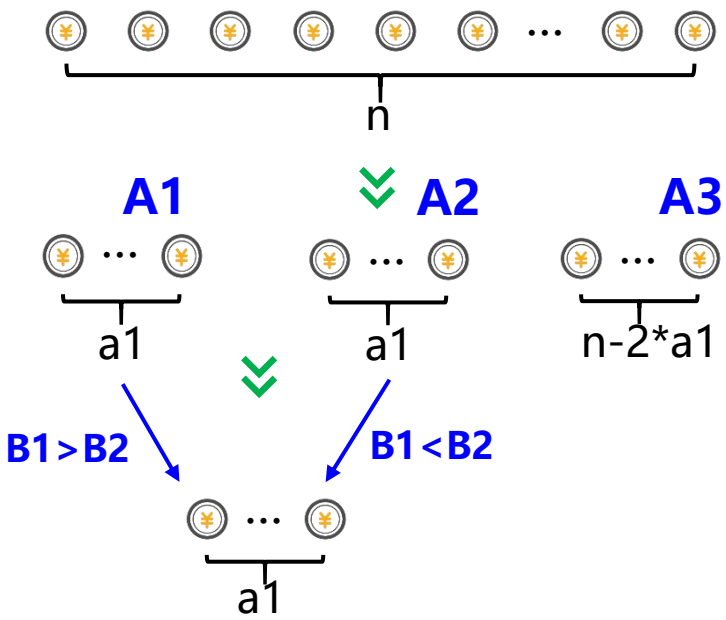
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

伪币寻找：三等分

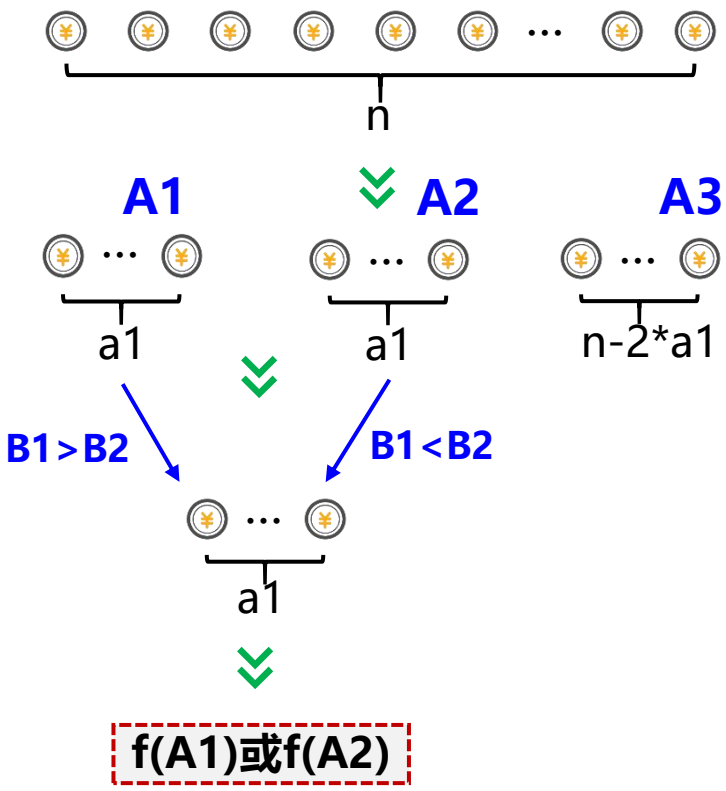
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

伪币寻找：三等分

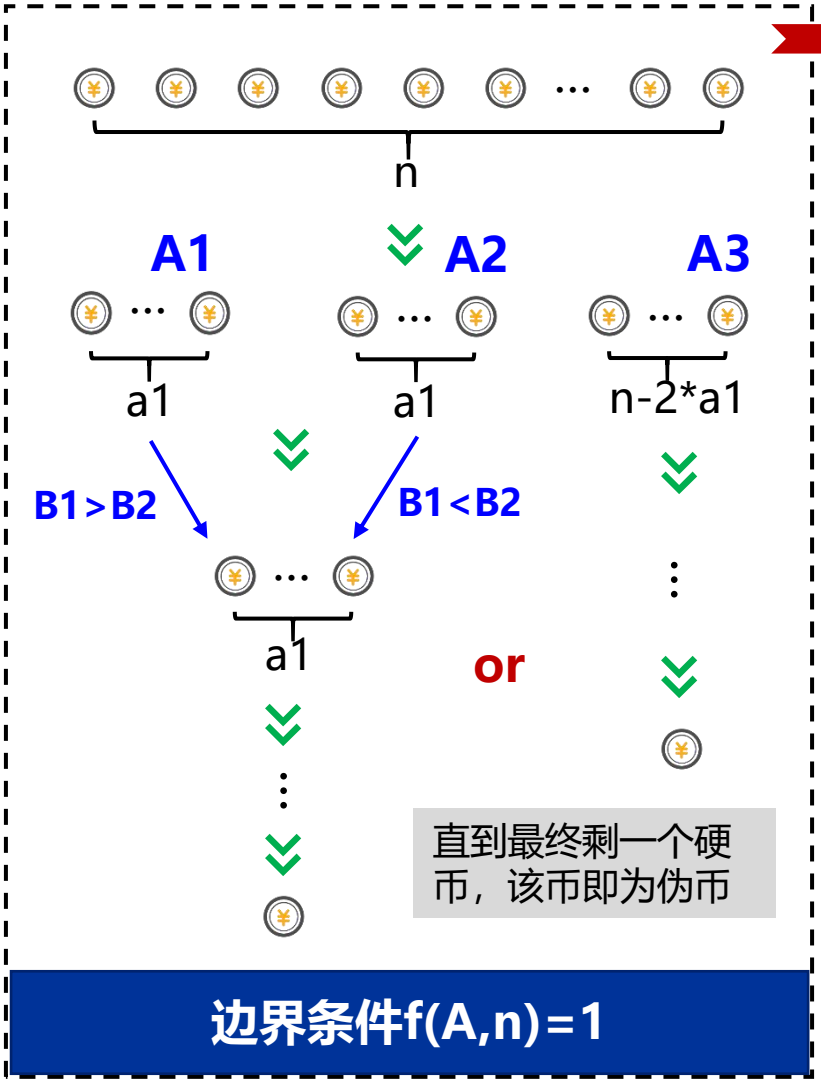
① 假设f(A)是能够实现从一群数量为n的硬币中找到伪币的函数，由于伪币质量较轻，可以建模为在数组A中找到值较小的那个




```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

治

伪币寻找：三等分边界条件




```
f(A,n)
{
        a1 = floor(n/3);//对数组进行三等分，不能整除时向下取整
        memcpy(A1,A,a1);//将A的前floor(n/3)个数组放到一起
        memcpy(A2,A+a1,a1);//将A的中间a1个数组放到一起
        memcpy(A3,A+2*a1,n-2*a1);//将A的剩余数组放到一起
        B1=sum(A1,a1);B2=sum(A2,a1); B3=sum(A3,n-2*a1);
        if(B1==B2)
                f(A3);
        elseif(B1<B2)
                f(A1);
        else
                f(A2);
}
```

14.1 算法思想（伪币问题）

参见课本：P434-P435

- **找出伪币**：给你一个装有15个硬币的袋子。15个硬币中有一个是伪造的，且**未知伪币比真币的重量情况**。你的任务是找出伪币。为了帮助你完成这一任务，将提供一台可用来比较两组硬币重量的仪器，利用这台仪器，可以知道两组硬币的重量是否相同。



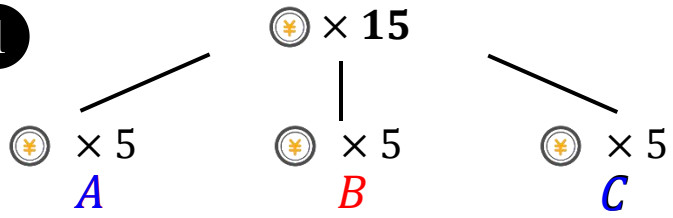
1. 判断：
   伪币比真币重 or
   伪币比真币轻
2. 找出伪币

14.1 算法思想（伪币问题）

参见课本：P434-P435

- **判断轻重：**给你一个装有15个硬币的袋子。15个硬币中有一个是伪造的，且未知伪币比真币的重量情况。你的任务是判断相比真币，伪币更轻还是更重（提供一台可用来比较两组硬币重量的天平）？

❶



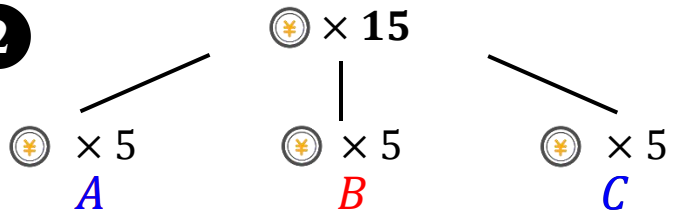
第一次比较

$A = B$ 则均为真币，伪币在C中。

第二次比较

$A > C$ 则伪币轻；
$A < C$ 则伪币重。

❷



第一次比较

$A \neq B$ 则存在伪币，C均为真币。

第二次比较

$A < C$ 则伪币轻；
$A > C$ 则伪币重；
$A = C$ 且 $A > B$则伪币轻；
且 $A < B$则伪币重。

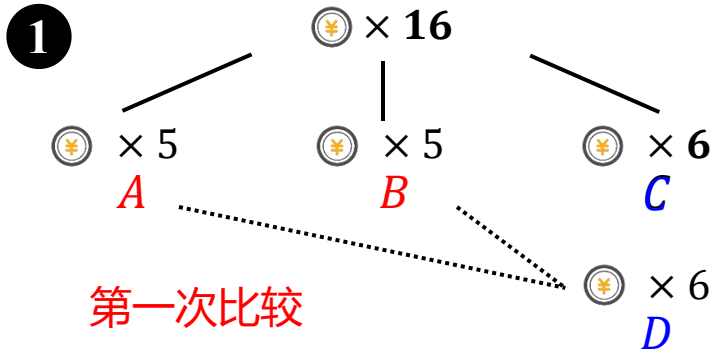
A 伪

B 伪

14.1 算法思想（伪币问题）

参见课本：P434-P435

- **判断轻重：**给你一个装有16个硬币的袋子。16个硬币中有一个是伪造的，且未知伪币比真币的重量情况。你的任务是判断相比真币，伪币更轻还是更重（提供一台可用来比较两组硬币重量的天平）？

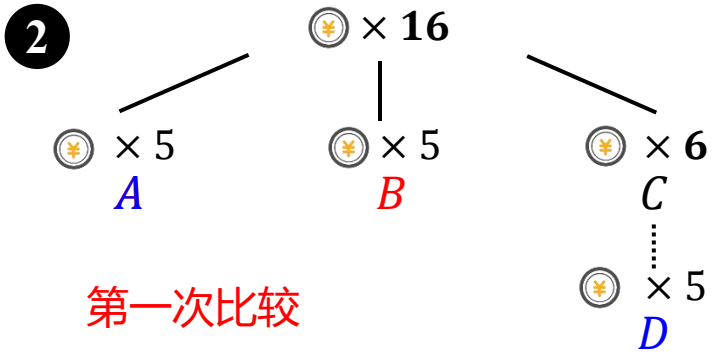


$A = B$ **则均为真币，伪币在C中；**

$A + B$归一堆，取6个作堆D。

$C > D$ **则伪币轻；**

$C < D$ **则伪币重。**



$A \neq B$ **则存在伪币，C均为真币；**

C中取5个作堆D。

A 伪

$A < D$ **则伪币轻；**

$A > D$ **则伪币重；**

$A = D$ **且** $A < B$ **则伪币重；**

且 $A > B$ **则伪币轻。**

B 伪

需要2次比较，A(B)数量= $\left\lfloor \frac{n}{3} \right\rfloor$

14.1 算法思想（伪币问题）

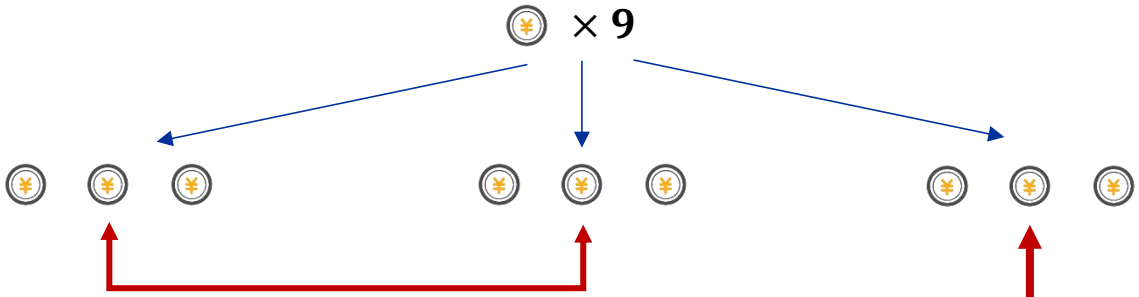
参见课本：P434-P435

- **例14-1 找出伪币：**给你一个装有9个硬币的袋子。9个硬币中有一个是伪造的，并且那个**伪币与真币的重量关系未知**。你的任务是找出这个伪造的硬币。为了帮助你完成这一任务，将提供一台可用来比较两组硬币重量的仪器，利用这台仪器，可以知道两组硬币的重量是否相同。

14.1 算法思想（伪币问题）

参见课本：P434-P435

- **例14-1 找出伪币：**给你一个装有9个硬币的袋子。9个硬币中有一个是伪造的，并且那个**伪币与真币的重量关系未知**。你的任务是找出这个伪造的硬币。为了帮助你完成这一任务，将提供一台可用来比较两组硬币重量的仪器，利用这台仪器，可以知道两组硬币的重量是否相同。



最后三枚硬币，在已知轻重时需要一次比较

14.1 算法思想（3分钟讨论）

参见课本：P434-P435

- **例14-1引申 找出伪币：**如果是 8 个硬币，重量未知如何 3 次完成?

- **例14-1引申 找出伪币：对于n个硬币**，重量未知，最多需多少次完成?

需要天平比较次数为：$[log_3(\text{硬币总数})]+1$

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题



考察重点



熟练掌握技巧

14.1 算法思想：分治法

- 分治法的设计思想就是把一个难以直接求解的大问题分割成一些规模较小的相同的问题，以便各个击破，分而治之
- 如果子问题都可解，并可利用这些子问题的解求出原问题的解，分治法就是可行的
- 反复应用分治手段，可以使子问题与原问题类型一致而其规模不断缩小，最终使子问题缩小到容易求出其解

14.1 算法思想：分治法

- 递归和分治像一对孪生兄弟，递归是分治策略的一种表达形式和实现方式。没有递归，分治就无从落地。
- 人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。
- 这种使子问题规模大致相等的做法是出自一种平衡(balancing) 子问题的思想，它几乎总是比子问题规模不等的做法要好。

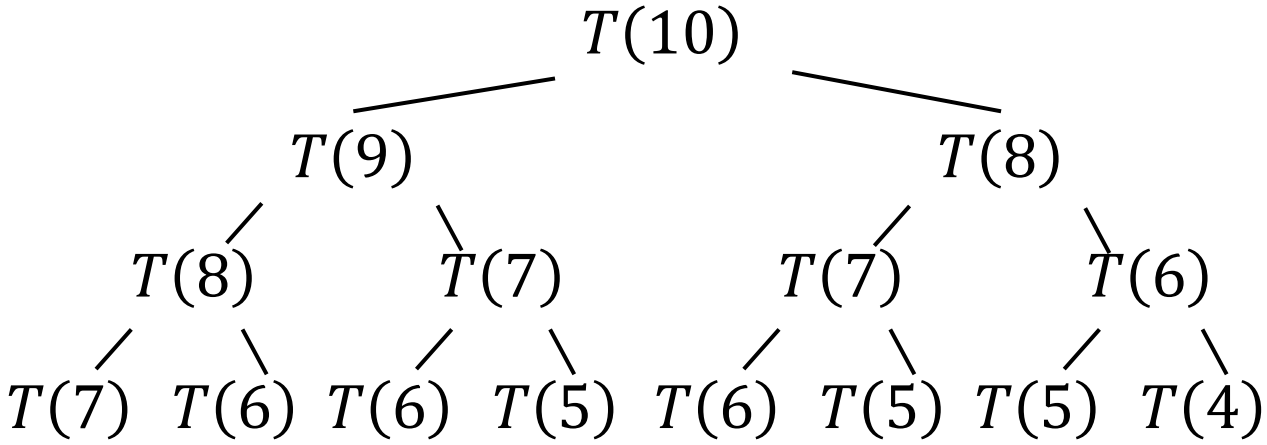
14.1 算法思想：分治法的特征

- 分治法所能解决的问题一般具有以下几个特征：
  - 该问题的规模缩小到一定的程度就可以容易地解决；
  - 该问题可以分解为若干个规模较小的相同问题；
  - 分解出的子问题的解可以合并为原问题的解；
  - 分解出的各个子问题是相互独立的。
    - 如果各子问题是不独立的，分治法要做许多不必要的工作，重复地解公共的子问题。

注：不独立的各子问题一般用动态规划较好

14.1 算法思想：分治法的特征

- 例如斐波那契数列：$T(n) = T(n-1) + T(n-2)$；
  - 这个在做树分解的时候 左右树分支有重复，假如n = 10，那么T9 需要求一次，T8需要求2次，T7需要求解3次....
  - 做了很多重复工作。



把一个大问题分割成一些规模较小的相同问题

1. **问题是否可以分解为若干个规模较小的相同问题?**
2. **问题的规模缩小到一定的程度是否可以很容易的解决?**

14.1 算法思想：分治法的步骤

- 在分治法中，我们递归地求解一个问题，在每层递归中应用如下三个步骤：
  - 分解(Divide)：将问题划分为一些子问题，子问题的形式与原问题一样，只是规模更小。
  - 求解(Conquer)：递归地求解出子问题。如果子问题的规模足够小，则停止递归，直接求解。
  - 合并(Combine)：将子问题的解组合成原问题的解。

14.1 算法思想：分治法的步骤

分治法的核心思路概括为分、治、合三点

简单的实例：

课题组两名老师，需要对100张期末考试的试卷按分数排列

分 **每名老师拿走50%试卷 或 老师甲工作效率高，两名老师按照7:3分配试卷**

治 **每名老师按照分数排好自己的试卷**

合 **针对于两组排好顺序的试卷进行合并排序**

14.1 算法思想：分治法与递归方程的关联

分治法的核心思路概括为分、治、合三点

$$T(n) = aT(n/b) + f(n)$$

分 每名老师拿走50%试卷 **或** 老师甲工作效率高，两名老师按照7:3分配试卷

两名老师

$$T(n) = 2T(n/2)$$

平均分配

$$T(n) = T(0.7n) + T(0.3n)$$

老师甲　　老师乙

a：分成了几个子问题?　　b：每个子问题的规模有多大?

14.1 算法思想：分治法与递归方程的关联

分治法的核心思路概括为分、治、合三点

$$T(n) = aT(n/b) + f(n)$$

合 针对于两组排好顺序的试卷进行合并排序

f（n）：分解 & 合并不同子问题时需要的额外成

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题



考察重点



熟练掌握技巧

14.1 算法思想：金块问题

参见课本：P435

- **例14-2 金块问题：**一个老板有一袋子金块。每个月有两名雇员会因其优异表现分别获得一个金块的奖励。按规矩，排名第一名的雇员得到最重的金块，排名第二的雇员得到最轻的金块。因为有新的金块定期加入袋中，所以每个月都必须找出最轻和最重的金块。假设有一台天平，希望用最少的比较次数找出最重和最轻的金块。



14.1 算法思想：金块问题

参见课本：P435

□ **例14-2 金块问题：**相当于在n个数中找出最大和最小的数。

直接求解

1. 找出最大值；
需要 n – 1 次比较。

2. 在剩下的 n – 1 个数中再找出最小值；
需要 n – 2 次比较。

金块问题　　伪代码

```
1.  min ← a[0]; max ← a[0];
2.  for i←1 to n-1 do
3.      if a[i] < min
4.          min ← a[i]
5.      else if a[i] > max
6.          max ← a[i]
```

共需 2n – 3 次比较

14.1 算法思想：金块问题

参见课本：P435

□ **例14-2 金块问题：**相当于在n个数中找出最大和最小的数。

分治法求解

1. 将金块分成两组 A 和 B
2. 分别找出 A 和 B 中最重和最轻的金块
3. 将第二步中找出的金块比较，得到最重的和最轻的金块

金块问题　　伪代码

```
1.  Max-min(A[0,n-1], max, min)
2.  if n<1 max ← min ← a[0], return;
3.     else  m ← n/2
4.        Max-min(A[0,m-1], max1, min1),
5.        Max-min(A[m,n-1], max2, min2),
6.        max ← max(max1, max2),
7.        min ← min(min1, min2),
8.        return.
```

14.1 算法思想：金块算法（3min 计算）

参见课本：P435

- 设 $c(n)$为使用分治法所需要的比较次数，有：

$$c(n)=\begin{cases}0 & \mathrm{n}=1\\ 1 & \mathrm{n}=2\\ 2\mathrm{c}(\mathrm{n}/2)+2 & \mathrm{n}=2^{\mathrm{k}}>2\end{cases}$$

- 根据主定理，$T(n) = \Theta(n)$

- 迭代展开可得到准确的比较次数 $\left(\frac{3n}{2}\right)-2$

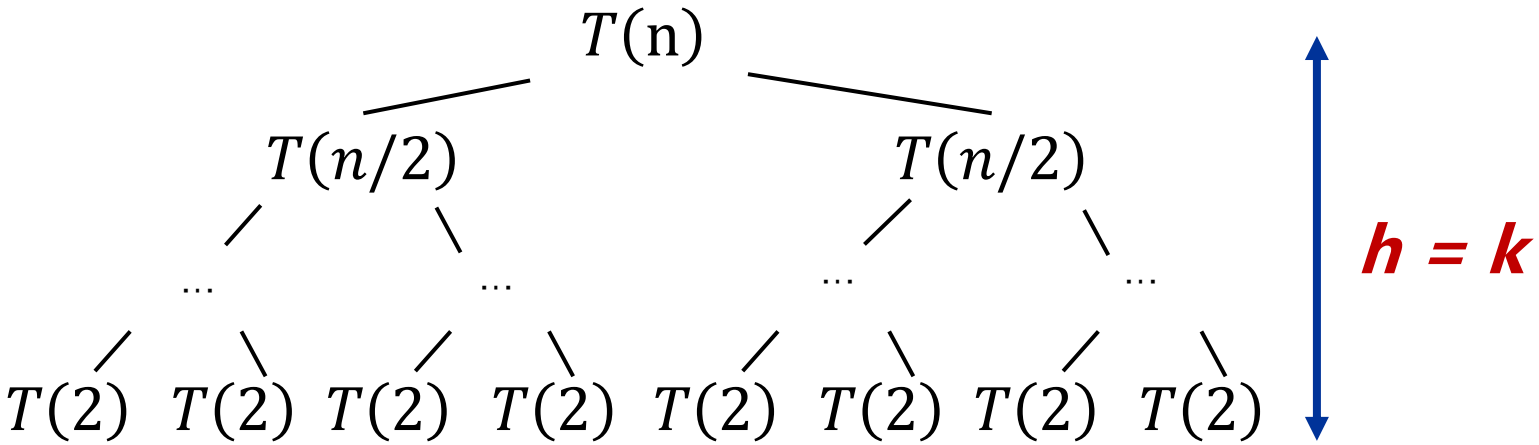
如何迭代展开?

14.1 算法思想：金块算法（3min 计算）

参见课本：P435

- 设 $c(n)$ 为使用分治法所需要的比较次数，有：

$$c(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ 2c(n/2)+2 & n=2^k>2\end{cases}$$

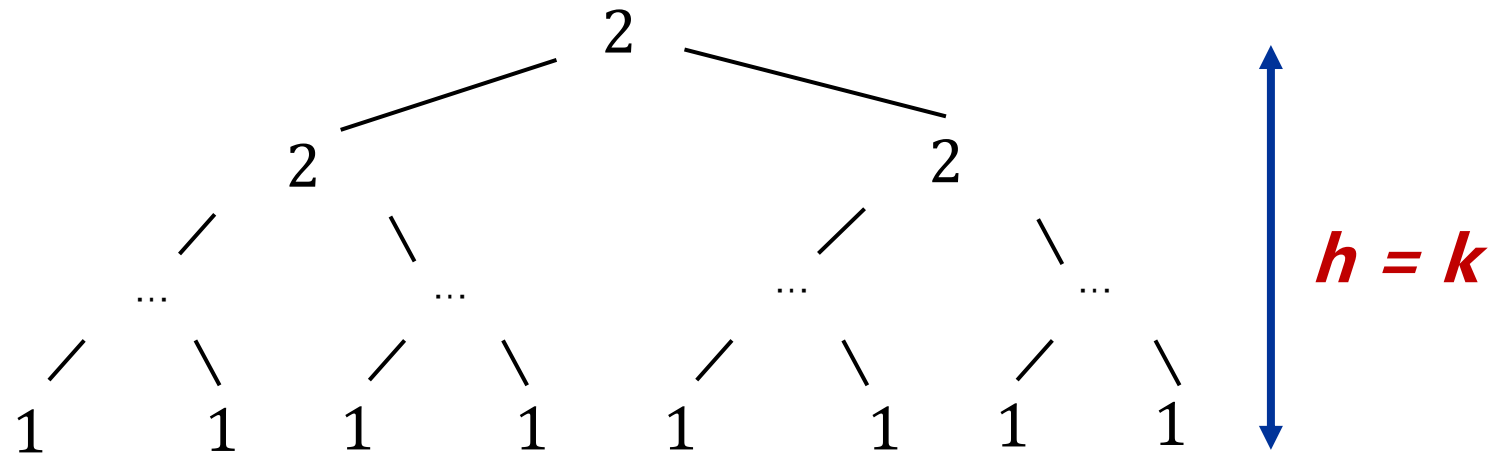


14.1 算法思想：金块算法（3min 计算）

参见课本：P435

- 设 $c(n)$ 为使用分治法所需要的比较次数，有：

$$c(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ 2c(n/2)+2 & n=2^k>2\end{cases}$$

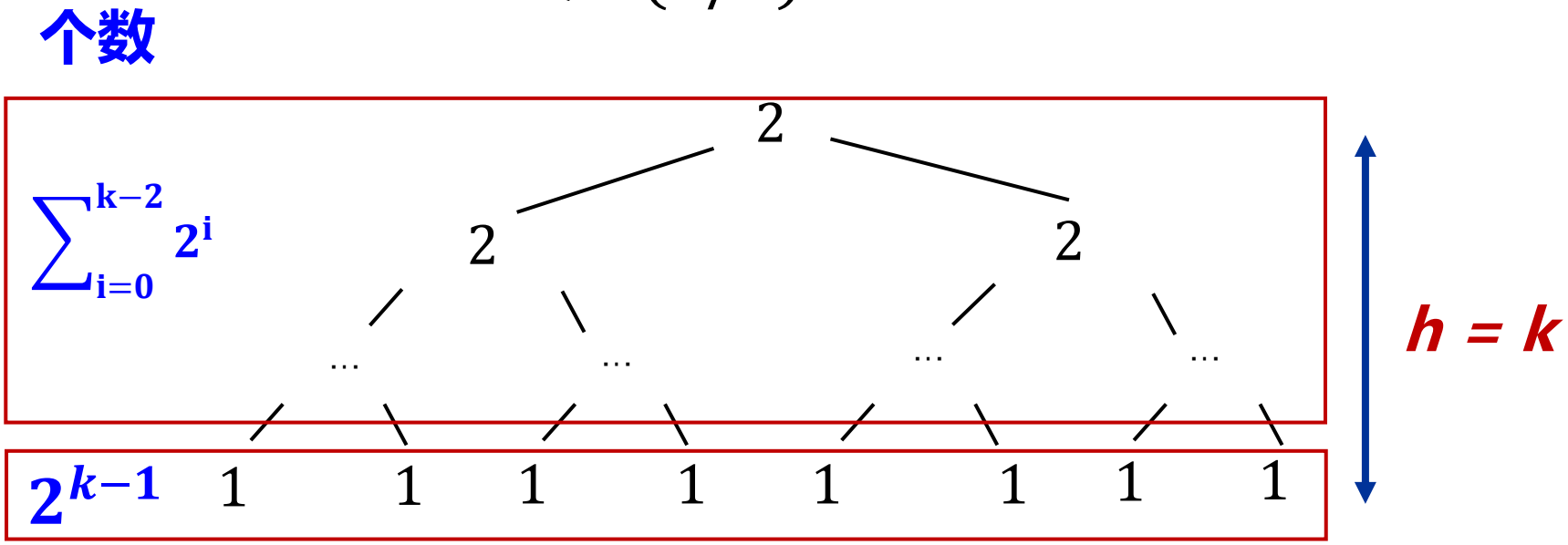


14.1 算法思想：金块算法（3min 计算）

参见课本：P435

□设 $c(n)$ 为使用分治法所需要的比较次数，有：

$$c(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ 2c(n/2)+2 & n=2^k>2\end{cases}$$



14.1 算法思想：金块算法（3min 计算）

参见课本：P435

- 设 $c(n)$ 为使用分治法所需要的比较次数，有：

$$c(n)=\begin{cases}0 & n=1\\ 1 & n=2\\ 2c(n/2)+2 & n=2^k>2\end{cases}$$

- 迭代展开可求得更准确的比较次数：

$$c(n)=2^{k-1}c(2)+\sum_{i=0}^{k-2}2^i\times 2=\left(\frac{3n}{2}\right)-2,\quad k=\log_2 n$$

- **分治法比直接求解法少用了25%次比较**

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题



考察重点



熟练掌握技巧

14.1 算法思想：矩阵相乘

参见课本：P435

- **例14-3 矩阵相乘：**两个n×n阶的矩阵A与B相乘得到一个n×n阶矩阵C，C可表示为：

$$C(i,j) = \sum_{k=1}^{n} A(i,k) * B(k,j),$$
$$1 \le i \le n,$$
$$1 \le j \le n,$$

- **传统方法：**

$$c_{ij} = \sum a_{ik} b_{kj} (1 \le k \le n)$$

  - 每计算 C 的一个元素 c_{ij}, 需要做 n 次乘法和 n – 1 次加法。
  - $T(n) = \Theta(n^3)$。

14.1 算法思想：矩阵相乘

参见课本：P435

- **例14-3 矩阵相乘：**两个n×n阶的矩阵A与B相乘得到一个n×n阶矩阵C，C可表示为：

$$C(i,j)=\sum_{k=1}^{n}A(i,k)\times B(k,j),$$
$$1\le i\le n,$$
$$1\le j\le n,$$

- **分治法：**
  - 将矩阵 A, B 和 C 中每一矩阵都分块成 4 个大小相等的子矩阵。由此可将方程 C = A · B 重写为：

$$\begin{bmatrix}C_{11} & C_{12}\\ C_{21} & C_{22}\end{bmatrix}=\begin{bmatrix}A_{11} & A_{12}\\ A_{21} & A_{22}\end{bmatrix}\cdot\begin{bmatrix}B_{11} & B_{12}\\ B_{21} & B_{22}\end{bmatrix}\Leftrightarrow\begin{cases}C_{11}=A_{11}B_{11}+A_{12}B_{21}\\ C_{12}=A_{11}B_{12}+A_{12}B_{22}\\ C_{21}=A_{21}B_{11}+A_{22}B_{21}\\ C_{22}=A_{21}B_{12}+A_{22}B_{22}\end{cases}$$

14.1 算法思想：矩阵相乘

参见课本：P435

- 从递归树可知：

$$T(n)=a^{h}T(1)+\sum_{k=0}^{h-1}a^{k}f\left(\frac{n}{b^{k}}\right)=a^{\log_b n}T(1)+\sum_{k=0}^{h-1}a^{k}f\left(b^{h-k}\right)$$

$$(h=\log_b a)$$

最底层
解决子问题的成本

分解和合并
子问题的成本

14.1 算法思想：矩阵相乘

参见课本：P435

口 分治法：

- 将矩阵 A、B 和 C 中每一矩阵都分块成 4 个大小相等的子矩阵。由此可将方程 $C = A \cdot B$ 重写为：

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \Leftrightarrow \begin{cases} C_{11} = A_{11}B_{11} + A_{12}B_{21} \\ C_{12} = A_{11}B_{12} + A_{12}B_{22} \\ C_{21} = A_{21}B_{11} + A_{22}B_{21} \\ C_{22} = A_{21}B_{12} + A_{22}B_{22} \end{cases}$$

子问题的基本单元

- 子问题的规模是n/2，重复8次：

$$T(n) = 8T(n/2) + f(n)$$

14.1 算法思想：矩阵相乘

参见课本：P435

口 分治法：

- 子问题的规模是n/2，重复8次：

$$T(n) = 8T(n/2) + f(n)$$

$f(n)$的求解方法?

- 看问题组合：4次加法?

分 合

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \Leftrightarrow \begin{cases} C_{11} = A_{11}B_{11} + A_{12}B_{21} \\ C_{12} = A_{11}B_{12} + A_{12}B_{22} \\ C_{21} = A_{21}B_{11} + A_{22}B_{21} \\ C_{22} = A_{21}B_{12} + A_{22}B_{22} \end{cases}$$

- 一个子矩阵相乘得一个n/2 x n/2 矩阵：实际上做了n/2 x n/2 x 4 次加法!

$$\boldsymbol{T(n) = 8T(n/2) + \Theta(n^2)}$$

14.1 算法思想：矩阵相乘

参见课本：P436

口 主定理：对形如$T(n) = aT(n/b) + f(n)$的递归式

$$T(n) = \begin{cases} \Theta\big(f(n)\big) & if\ f(n) = \Omega(n^{log_b a+\epsilon}) \quad ① \\ \Theta(n^{log_b a} log_n) & if\ f(n) = \Omega(n^{log_b a}) \quad ② \\ \Theta(n^{log_b a}) & if\ f(n) = \Omega(n^{log_b a+\epsilon}) \quad ③ \end{cases}$$

- $T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2) = \Theta(n^3)$
- **对策：要么减小 a，要么增大 b （递归方程的优化核心）**
- $f(n)$ 目前不占主导因素，暂不需优化。

14.1 算法思想：Strassen 方法

参见课本：P436

- 为了降低复杂度，在不改变矩阵分块的情况下，可以减小递归函数前的系数:

$$
\begin{aligned}
M_1 &= A_{11}(B_{12}-B_{22}) \\
M_2 &= (A_{11}+A_{12})\,B_{22} \\
M_3 &= (A_{21}+A_{22})\,B_{22} \\
M_4 &= A_{22}(B_{21}-B_{11}) \\
M_5 &= (A_{11}+A_{22})(B_{11}+B_{22}) \\
M_6 &= (A_{12}-A_{22})(B_{21}+B_{22}) \\
M_7 &= (A_{11}-A_{21})(B_{11}+B_{12})
\end{aligned}
\quad\Leftrightarrow\quad
\begin{aligned}
C_{11} &= M_5+M_4-M_2+M_6 \\
C_{12} &= M_1+M_2 \\
C_{21} &= M_3+M_4 \\
C_{22} &= M_5+M_1-M_3-M_7
\end{aligned}
$$

- $T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2) = \Theta\left(n^{\log 7}\right) \approx \Theta(n^{2.81})$

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题

 **考察重点**  **熟练掌握技巧**

14.1 算法思想：大整数乘法

- 3278 × 41926：

```
          4 1 9 2 6
    ×         3 2 7 8
-------------------------
        3 3 5 4 0 8
      2 9 3 4 8 2
        8 3 8 5 2
    1 2 5 7 7 8
-------------------------
  1 3 7 4 3 3 4 2 8
```

- 小学方法计算a × b，用 b 中的每一位依次乘 a，并将结果逐行排列，按 b 的相应位对齐再相加。
- 整个计算称为**长乘**，b 中的一位乘 a 称为**短乘**。
- 两个 n 位数相加，需要 n 次加法。
- 一个 n 位数的短乘，需要 n 次乘法和 n 次加法。

14.1 算法思想：大整数乘法

- 3278 × 41926：

```
          4 1 9 2 6
  ×         3 2 7 8
-------------------
        3 3 5 4 0 8
      2 9 3 4 8 2
        8 3 8 5 2
    1 2 5 7 7 8
-------------------
  1 3 7 4 3 3 4 2 8
```

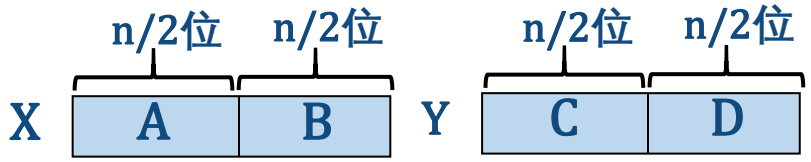
- 两个 n 位数的长乘，需要 n 个短乘和 n-1 次加法。这里的加法因为进位和对位的关系，中间结果位数不会超过 2n。所以总的基本操作次数是：$n \times 2n+(n-1) \times 2n=4n^2-2n$ 次。
- 假如有两个百万位的整数相乘，大概需要**4万亿次**基本操作。

大整数的乘法的复杂度是$O(n^2)$

14.1 算法思想：大整数乘法

□ 分治法：

- 分：将X和Y分别拆解成AB和CD
- 治：分别计算AC，AD，BC，BD
- 合：利用公式具体式子，计算剩余部分



$$XY=(A*10^{\frac{n}{2}}+B)(C*10^{\frac{n}{2}}+D)$$

合

分

$$=AC*10^{n}+(AD+BC)*10^{\frac{n}{2}}+BD$$

$$XY=(A*10^{\frac{n}{2}}+B)(C*10^{\frac{n}{2}}+D)$$

$$=AC*10^{n}+(AD+BC)*10^{\frac{n}{2}}+BD$$

$$T(n)=\begin{cases}O(1) & n=1\\ 4T(n/2)+O(n) & n>1\end{cases}$$

根据主定理，T(n) = $O(n^2)$

如何降低复杂度？
优化方向是什么？

14.1 算法思想：大整数乘法-Karatsuba算法

口 算法分析

$$T(n)=\begin{cases} O(1) & n=1 \\ 4T(n/2)+O(n) & n>1 \end{cases}$$

- f(n) 的阶数更低，因此我们要优化子问题的个数（也就是4）

将两个大整数表达如下：

$$XY = AC*10^{n} + ((A-B)(D-C) + AC + BD)*10^{\frac{n}{2}} + BD$$

或者：

$$XY = AC*10^{n} + ((A+B)(C+D) - AC - BD)*10^{\frac{n}{2}} + BD$$

这样，子问题的个数从原有的 4 个变成 3 个，时间复杂度函数为：

$T(n)=3T(n/2)+O(n)=\mathbf{O(n^{\log 3})\approx O(n^{1.59})}$

14.1 算法思想：大整数乘法-Karatsuba算法

- **细节问题：**两个 X·Y的复杂度都是 $O(n^{\log 3})$，但考虑到 A + C 和 B + D 两个数相加可能进位变成 m + 1 位，使问题的规模变大，故不选择第 2 种方案。
- 该算法是 Anatolii Alexeevitch Karatsuba 博士在1960年提出的。
- 两个百万位的整数相乘，小学解法需要 1 万亿次乘法，而 Karatsuba 方法“仅” 需要 35 亿次，是长乘代价的 **1/287**。

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题

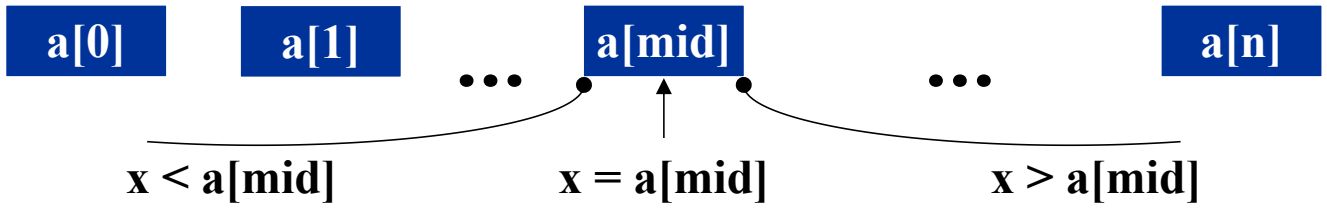
 **考察重点**  **熟练掌握技巧**

14.1 算法思想：二分搜索算法

- **二分搜索:** 给定已按升序排好序的 n 个元素 a[0:n-1]，现要在这 n 个元素中找出一特定元素 x。
  - 只要子问题规模缩小到一定的程度，就容易解决。比如 n = 1 时，只要比较此元素和 x 就可以确定 x 是否在表中。
  - 问题可分解为两个规模折半的相同的搜索问题。
  - 两个搜索问题子问题是相互独立的。
  - 子问题的解就是原问题的解。
- 结论：适合用分治策略求解。

14.1 算法思想：二分搜索算法

- **二分搜索:** 每次递归计算 a 的中间元素 a[mid] 并与 x 比较。
  - 若 x = a[mid]，则 x 在 L 中的位置就是 mid。
  - 如果 x < a[mid]，因为 a 是递增排序的，假如 x 在 a 中的话，x 必然排在 a[mid] 的前面，所以在 a[mid] 的前面查找 x 即可。
  - 如果 x > a[mid]，同理在 a[mid] 的后面查找 x 即可。



- 无论是在前面还是后面查找 x，其方法都和在 a 中查找 x 一样，只不过是查找的规模缩小了。

14.1 算法思想：二分搜索算法

- **二分搜索:** 每次递归计算 a 的中间元素 a[mid] 并与 x 比较。
- 二分搜索算法伪代码：

```
Template<class Type>
Int BinarySearch(Type a[], const Type&x, int l, int r)
{
      while (r>=1)
           {
              int m=(1+r)/2;
              if (x==a[m]) return m;
              if (x<a[m]) r=m-1; else l=m+1;
           }
}
```

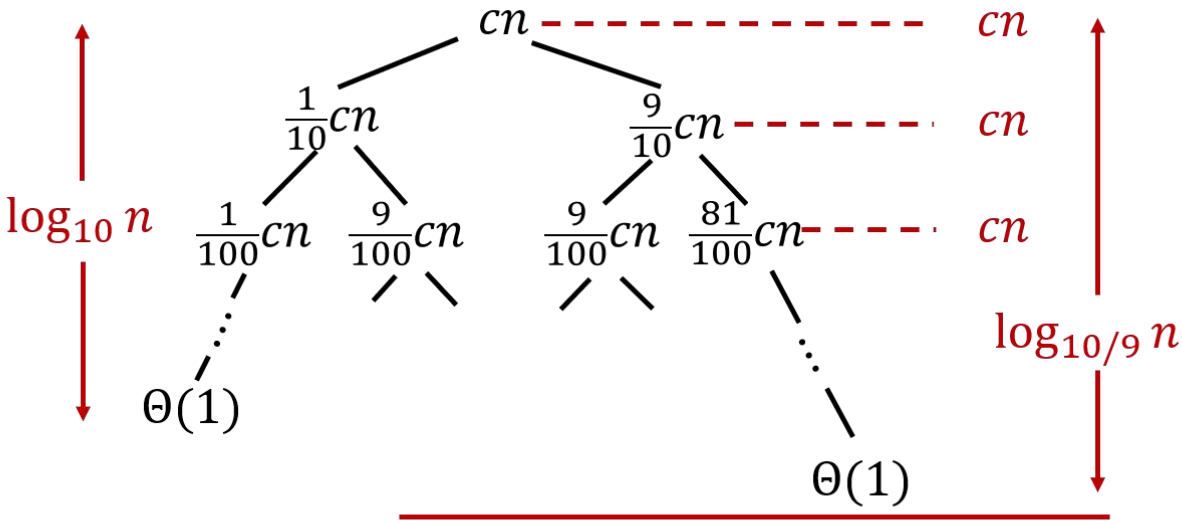
14.1 算法思想：二分搜索算法

- **二分搜索:** 每次递归计算 a 的中间元素 a[mid] 并与 x 比较。
  - 每执行一次算法的 while 循环，待搜索数组的大小减少一半。
  - 在最坏情况下，while循环被执行了 $O(logn)$次。
  - 循环体内运算需要 $O(1)$ 时间。
  - 整个算法在最坏情况下的计算时间复杂性为 $O(logn)$。

Q: 一定要二分吗？黄金分割怎么样?

14.1 算法思想：二分搜索算法

- **递归树展开:** 展开 $T(n) = T(0.1n) + T(0.9n) + \Theta(n)$ 的递归树并计算递归树的深度和 T(n) 的渐近值。



$$cnlog_{10}n \le T(n) \le cn\log_{10/9} n + O(n)$$

- 结论：树的深度取决于比例降低的更慢的一侧。

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想
- 设计方法与实例
  - 金块问题
  - 矩阵相乘
  - 大整数乘法
  - 二分搜索

□ 分治法的应用实例

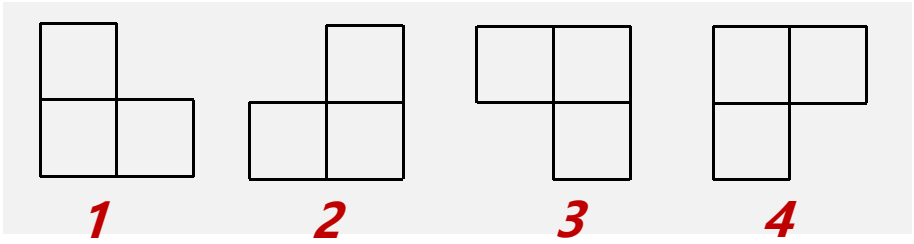
- 棋盘覆盖
- 循环赛日程表
- 归并排序
- 快速排序
- 寻找第k小
- 平面点对
- 平面点集凸包问题

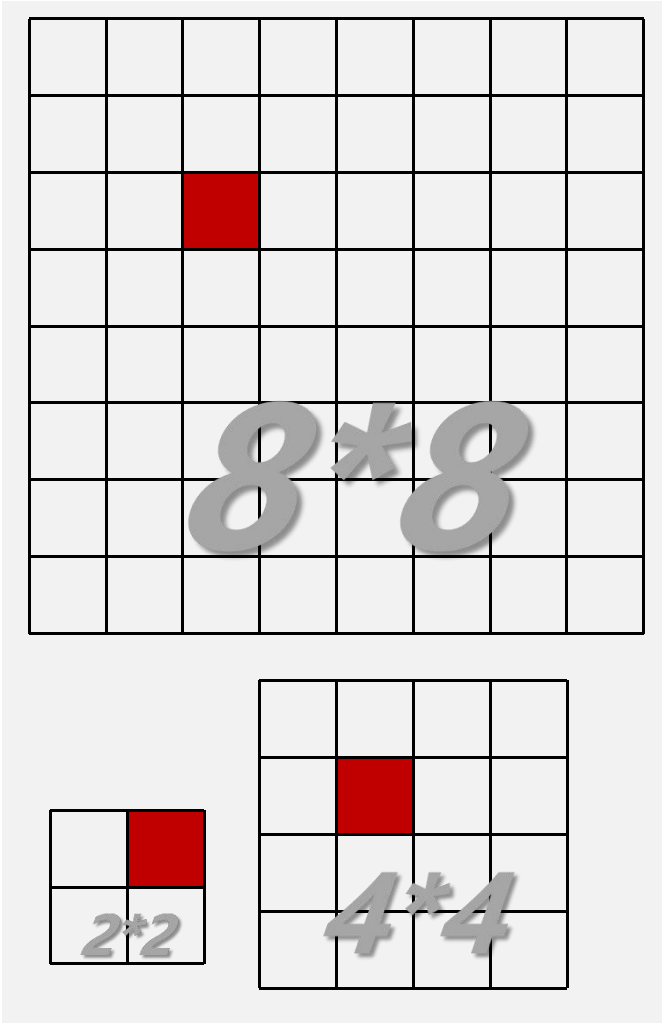
考察重点　　熟练掌握技巧

14.2.1 残缺棋盘

参见课本：P440

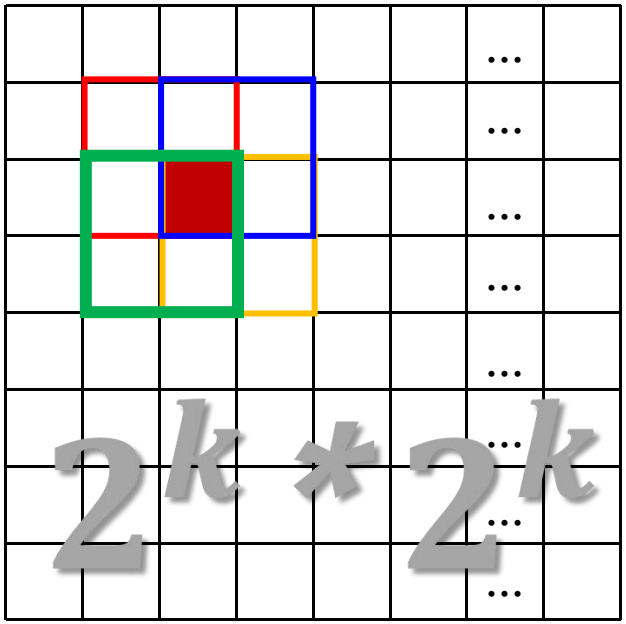
- 在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一**特殊方格**，且称该棋盘为一**特殊棋盘**。
- 在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖。

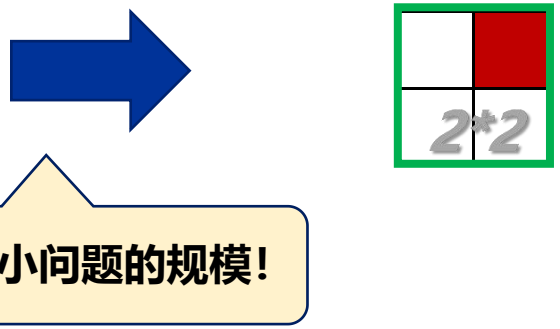




14.2.1 残缺棋盘

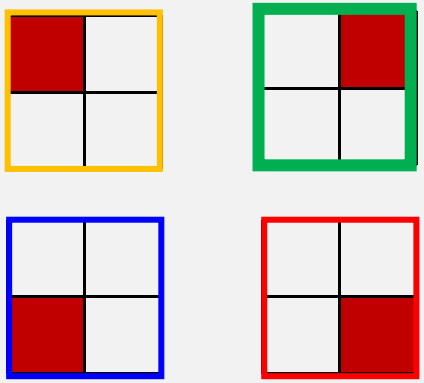
参见课本：P440





- 结论：当问题的规模缩小到一定程度时就可以很容易地解决！

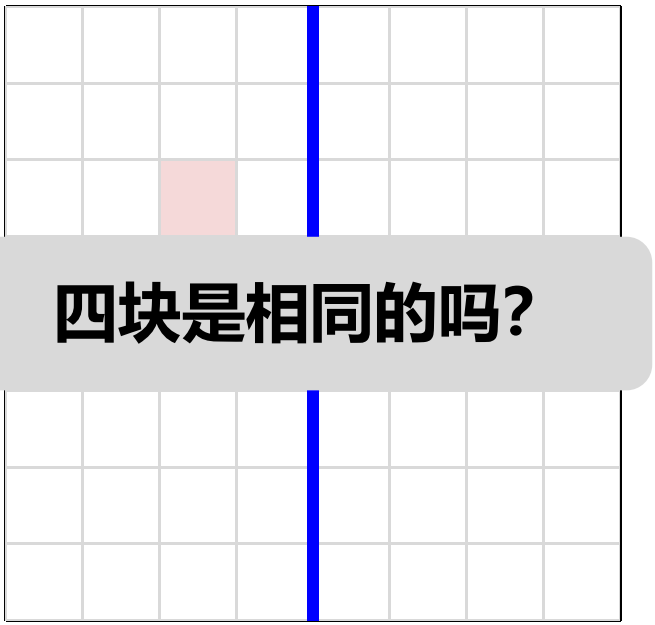
- 如果是一个2*2的方格，我们必定可以用一个L型的骨牌填满



14.2.1 残缺棋盘

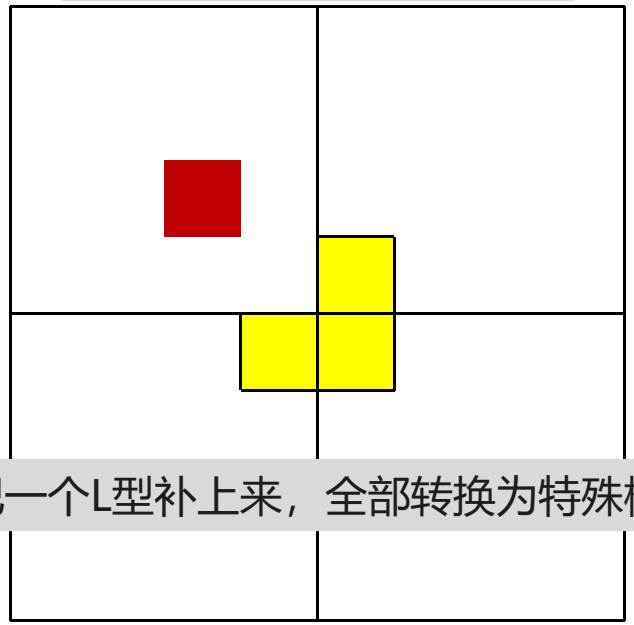
参见课本：P441

Q：如何让问题的规模缩小到2*2?



四块是相同的吗?

只有一块棋盘有特殊方格



把一个L型补上来，全部转换为特殊棋盘

14.2.1 残缺棋盘

参见课本：P441

- 解决流程

1. 当 k > 0 时，将 $2^k \times 2^k$棋盘分割为 4 个 $2^{k-1} \times 2^{k-1}$ 子棋盘如上张PPT左图所示。

   特殊方格必位于 4 个较小的子棋盘之一中，其余 3 个子棋盘中无特殊方格

2. 为了将这 3 个无特殊方格的子棋盘转化为特殊棋盘，可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处

   从而将原问题转化为 4 个较小规模的棋盘覆盖问题

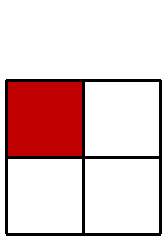
3. 递归地使用这种分割，直至棋盘简化为棋盘 2×2

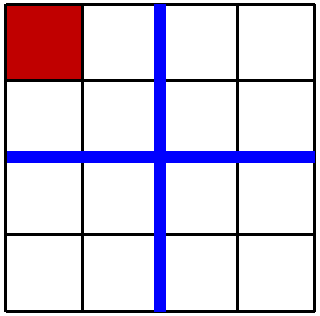
14.2.1 残缺棋盘

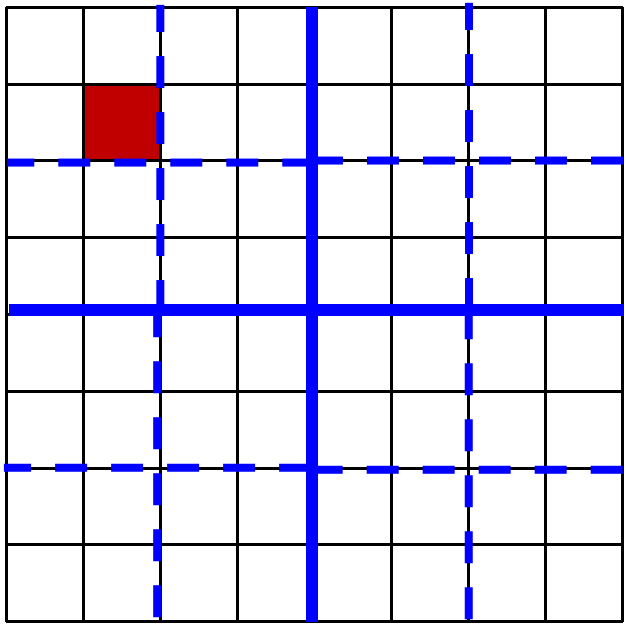
参见课本：P441

➢ 需要L型骨牌树与棋盘大小的关系

棋盘规模	所需L型骨牌数	等效换算数
2*2	1	4^0
4*4	4×1+1	4^1+4^0
8*8	4× (4×1+1)	$4^2+4^1+4^0$
…	…	…
$2^k * 2^k$	$(4^k-1)/3$	$\sum_{0\le i\le k-1}4^i$







14.2.1 残缺棋盘

参见课本：P441

- 递归方程如下：

$$T(k)=\begin{cases} O(1) & k=0 \\ 4T(k-1)+O(1) & k>1 \end{cases} =4^k T(0)+O(4^{k-1})=O(4^k)$$



亦即，把一个有$n=4^k$个方格组成棋盘全部覆盖，需要(n-1)/3个 L 型骨牌，时间复杂度为：

$$T(n)=4T(n/4)+c=O(n).$$

14.2.1 残缺棋盘

参见课本：P442

```
void chessBoard(int tr, int tc, int dr, int dc, int size)
  {
    if (size == 1) return;
       int t = tile++,  // L型骨牌个数
     s = size/2;  // 子棋盘大小
    // 覆盖左上角子棋盘
    if (dr < tr + s && dc < tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr, tc, dr, dc, s);
    else {// 此子棋盘中无特殊方格
      // 用 L型骨牌覆盖右下角
      board[tr + s - 1][tc + s - 1] = t;
      // 覆盖其余方格
      chessBoard(tr, tc, tr+s-1, tc+s-1, s);}
    // 覆盖右上角子棋盘
    if (dr < tr + s && dc >= tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr, tc+s, dr, dc, s);
    else {// 此子棋盘中无特殊方格
      // 用L型骨牌覆盖左下角
      board[tr + s - 1][tc + s] = t;
      // 覆盖其余方格
     chessBoard(tr, tc+s, tr+s-1, tc+s, s);}
      // 覆盖左下角子棋盘
    if (dr >= tr + s && dc < tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr+s, tc, dr, dc, s);
    else {// 用 L型骨牌覆盖右上角
      board[tr + s][tc + s - 1] = t;
      // 覆盖其余方格
      chessBoard(tr+s, tc, tr+s, tc+s-1, s);}
    // 覆盖右下角子棋盘
    if (dr >= tr + s && dc >= tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr+s, tc+s, dr, dc, s);
    else {// 用 L型骨牌覆盖左上角
      board[tr + s][tc + s] = t;
      // 覆盖其余方格
      chessBoard(tr+s, tc+s, tr+s, tc+s, s);}
}
void OutputBoard(int size){
    for (int i = 0; i < size; i++){
        for (int j = 0; j < size; j++)
            cout << setw (5) << Board[i][j];
            cout << endl;
    }
}
```

14.2.1 残缺棋盘

参见课本：P442

```
void chessBoard(int tr, int tc, int dr, int dc, int size)
  {
    if (size == 1) return;
      int t = tile++,  // L型骨牌个数
     s = size/2;  // 子棋盘大小
    // 覆盖左上角子棋盘
if (dr < tr + s && dc < tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr, tc, dr, dc, s);
   else {// 此子棋盘中无特殊方格
      // 用 L型骨牌覆盖右下角
      board[tr + s - 1][tc + s - 1] = t;
      // 覆盖其余方格
      chessBoard(tr, tc, tr+s-1, tc+s-1, s);}
    // 覆盖右上角子棋盘
    if (dr < tr + s && dc >= tc + s)
      // 特殊方格在此子棋盘中
      chessBoard(tr, tc+s, dr, dc, s);
    else {// 此子棋盘中无特殊方格
      // 用L型骨牌覆盖左下角
      board[tr + s - 1][tc + s] = t;
      // 覆盖其余方格
     chessBoard(tr, tc+s, tr+s-1, tc+s, s);}
      // 覆盖左下角子棋盘
```

残缺方块所在行

残缺方块所在列

棋盘中左上角方格所在行

棋盘中左上角方格所在列

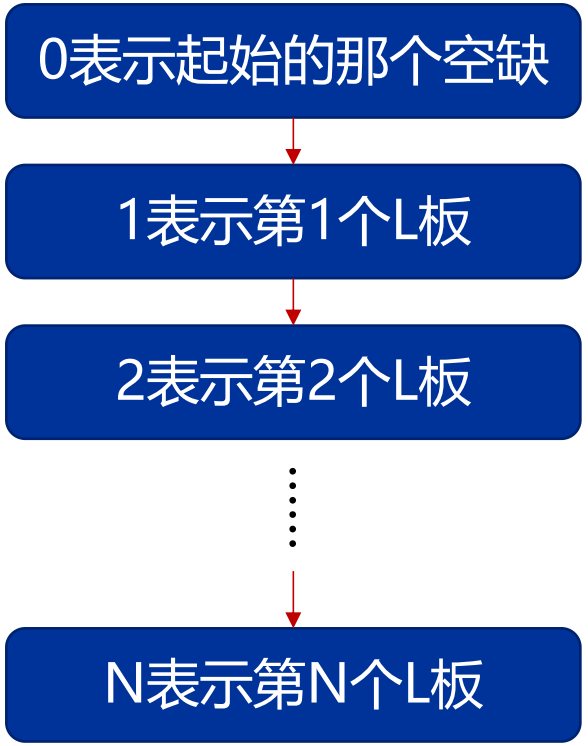
有特殊方格继续划分，无特殊方格采用上述的**补方格法**

14.2.1 残缺棋盘

参见课本：P442

- 该方法的输出图示

3	3	4	4	8	8	9	9
3	2	2	4	8	7	7	9
5	2	6	0	10	10	7	11
5	5	6	6	1	10	11	11
13	13	14	1	1	18	19	19
13	12	14	14	18	18	17	19
15	12	12	16	20	17	17	21
15	15	16	16	20	20	21	21



目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

 考察重点  熟练掌握技巧

□ 分治法的应用实例



棋盘覆盖

- 循坏赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题

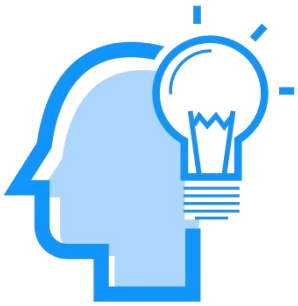
补充：循环赛日程表

- **设有n=2k个运动员要进行网球循环赛，现在要设计一 个满足以下要求的比赛日程表：**

  (1) 每个选手必须与其他 个选手各赛一次

  (2) 每个选手一天只能赛一次

  (3) 循环赛一共进行 n-1 天



可以将比赛表设计成一个 **n 行 n-1 列**的二维表，其中**第 i 行第 j 列**的元素表示和第 i 个选手在第 j 天比赛的选手号。

补充：循环赛日程表

- 按**分治策略**，可以将所有的选手分为两半，n个选手的比赛日程表就可以通过为n/2个选手设计的比赛日程表来决定。
- 递归地使用这种**一分为二**的策略对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单了。
- 这时只要让这两个选手进行比赛就可以了。

只有两个选手的日程安排：

1	2
2	1

只有四个选手的日程安排：

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

补充：循环赛日程表

- 右表是8个选手的比赛日8个选手的比赛日程表程表。
- **左上角**与**左下角**的2小块分别为**选手1至选手4**以及**选手5至选手8**前3天的比赛日程。
- 将左上角小块的数字复制到右下角，左下角的数字复制到右上角，就排好了选手们后4天的比赛日程

八个选手的比赛日程表

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

补充：循环赛日程表——代码

```
void gametable(int k)
{
    int a[100][100];
    int n, temp, i, j, p, t;
    n=2;//两个参赛选手日程
    a[1][1]=1;a[1][2]=2;
    a[2][1]=2;a[2][2]=1;
    for(t=1; t<k; t++)//迭代处理，依次处理2^n....2^k个选手的比赛日程
    {
        temp=n; n=n*2;//填左下角元素
        for(i=temp+1; i<=n;i++)
            for(j=1; j<=temp; j++)
                a[i][j]=a[i-temp][j]+temp;
        //左下角和左上角元素的对应关系
        for(i=1;i<=temp;i++)
        //将左下角元素抄到右上角
            for(j=temp+1;j<=n;j++)
                a[i][j]=a[i+temp][(j+temp)%n];
        for(i=temp+1;i<=n;i++)
        //将左上角元素抄到右下角
            for(j=temp+1;j<=n;j++)
                a[i][j]=a[i-temp][jtemp];
    }
}
```

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题



考察重点

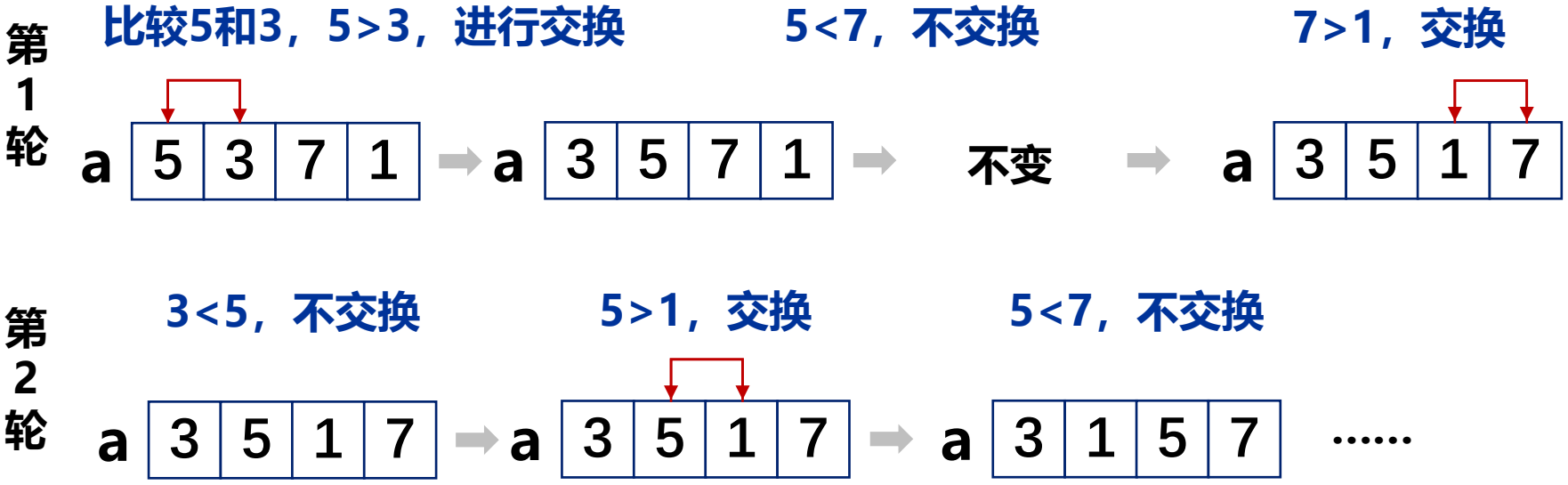


熟练掌握技巧

冒泡排序

参见课本：P040

- **冒泡排序：**冒泡排序是另一种简单的排序方法。在冒泡过程中，对两两相邻的元素进行比较，如果左边的元素大于右边的，则交换两个元素，将最大的元素逐步移到最右侧。每执行1次冒泡过程，元素比较的次数为**n-1**。



冒泡排序

参见课本：P040

```
template <class T>
void bubble(T a[], int n)
{// Move the largest element of a[0:n-1] to right.
      for (int i = 0; i < n–1; i++)
            if (a[i] > a[i+1])
                  Swap(a[i], a[i+1]);}
template <class T>
void bubbleSort(T a[], int n)
{// Sorted a[0:n–1]
      for (int i = n; i >1; i--)
            bubble(a, i);
}
```

- 实例特征：n
- **选定的操作：比较次数。**
- 一次冒泡的比较次数是n–1
- 排序中循环的深度是n
- T=Σ(n–1)=(n–1)n/2
- 算法的复杂度为$O(n^2)$

14.2.2 归并排序

参见课本：P443

- **归并排序：** 老师要求按照分数递减方式给试卷排序。两位助教各自负责一半试卷整理任务。两位助教整理好各自负责试卷后，应该如何整合所有试卷，保证所有试卷以分数递减方式排序。
  - 首先将两张卷最上面的那两张相互比较，将较大的那张放在整合的试卷堆中；

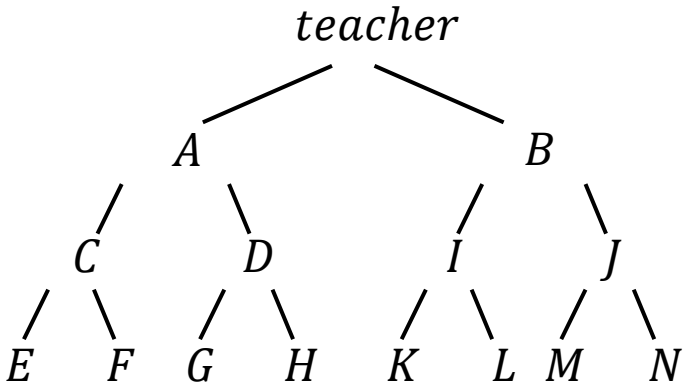


  - 重复比较步骤，直至所有试卷排序完毕；
  - 时间复杂度为$O(n)$。

14.2.2 归并排序

参见课本：P443

- **归并排序：** 老师要求按照分数递减方式给试卷排序。两位助教各自负责一半试卷整理任务。两位助教整理好各自负责试卷后，应该如何整合所有试卷，保证所有试卷以分数递减方式排序。
  - 如果助教A(B)选择让C和D外包工作，而C(D)也选择让E和F外包，如此类推….



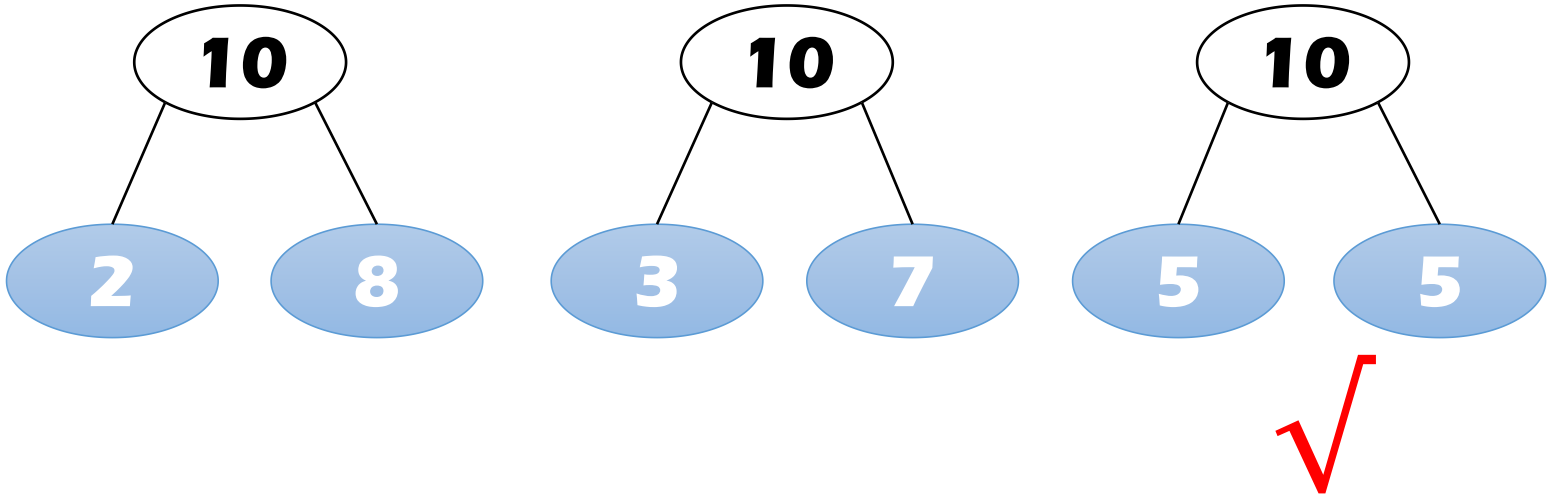


14.2.2 归并排序

参见课本：P443

- **归并排序：** 老师要求按照分数递减方式给试卷排序。两位助教各自负责一半试卷整理任务。两位助教整理好各自负责试卷后，应该如何整合所有试卷，保证所有试卷以分数递减方式排序。

Q: 每次分给的两个人任务如何划分?



14.2.2 归并排序

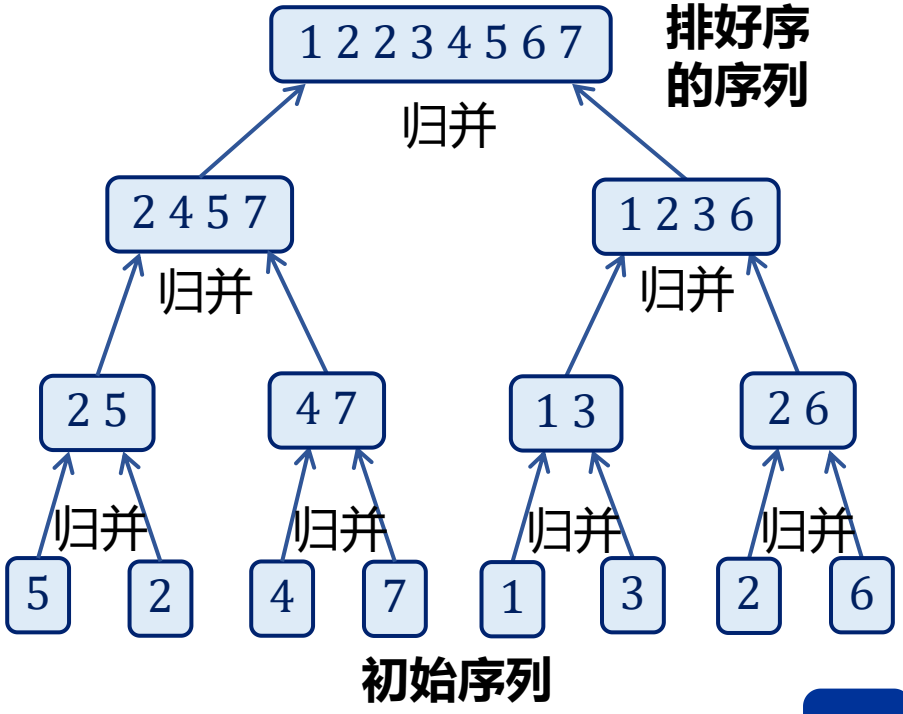
参见课本：P443

口 归并排序(A, p, r)：对序列A进行排序。

算法求解

1. 如果 n=1，直接排序
2. 否则将问题二等分：
   p=⌊ n/2⌋
   MERGE-SORT A[1..p]
   MERGE-SORT A[p+1..n]
3. 最后合并：
   MERGE the 2 sorted lists

图解



14.2.2 归并排序

参见课本：P443

```
template <class T>
void sort(T E, int n)
{// 对E中的n个元素进行排序，k为全局变量
      if (n >= k){
         i = n/k;
         j = n-i;
         // 令A包含E中的前i个元素，B包含E中剩余的j个元素
         sort(A, i);
         sort(B, j);
         merge(A, B, E, i, j); // 把A和B合并到E
         }
}
```

14.2.2 归并排序

参见课本：P443

- 设$t(n)$为归并排序算法，则有以下递推公式：

$$t(n)=\begin{cases} d & n<k \\ t\left(\frac{n}{k}\right)+t\left(n-\frac{n}{k}\right)+cn, & n\geq k \end{cases}$$

  - 当$n/k \approx n-(n/k)$时$t(n)$的值最小（balance 原理）；
  - 当$k=2$时，分治法通常具有最佳性能；
  - 当$k>2$时递归展开的深度为$\log_a n, a=k/(k-1)$，超过$\log_2 n$。

14.2.2 归并排序

参见课本：P443

- 当$k = 2$时，则有以下递推公式：

$$t(n) = \begin{cases} d & n \le 1 \\ t\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + t\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn, & n > 1 \end{cases}$$

- 算法复杂度为 $O(nlogn)$：

$$T(n) = \begin{cases} \Theta\left(f(n)\right) & if\ f(n) = \Omega(n^{log_b a + \epsilon}) \quad ① \\ \Theta(n^{log_b a} log_n) & if\ f(n) = \Omega(n^{log_b a}) \quad ② \\ \Theta(n^{log_b a}) & if\ f(n) = \Omega(n^{log_b a + \epsilon}) \quad ③ \end{cases}$$

14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

1	3	4	2	3	6

分析与理解问题：有几个逆序对?

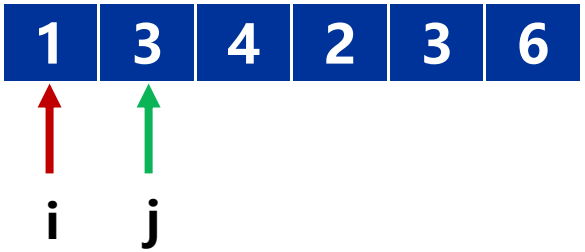
三个逆序对：(3,2) (4,2) (4,3)

问题：如何求解逆序对的个数?

14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

解决思路一：循环比较的方法




```
for (i = 1; i < n; i ++)
    for (j = i + 1; j < n; j ++)
        逐个进行比较，计算
    end
end
```

时间复杂度：$O(n^2)$

14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

解决思路二：分治法

思考：如果把待计算序列分为两个子序列，可以递归地计算两个子序列的逆序对数，归并的时候还要计算两个子序列之间的逆序对数

分 计算序列分为两个子序列

治 分别计算两个**子序列的逆序对数**

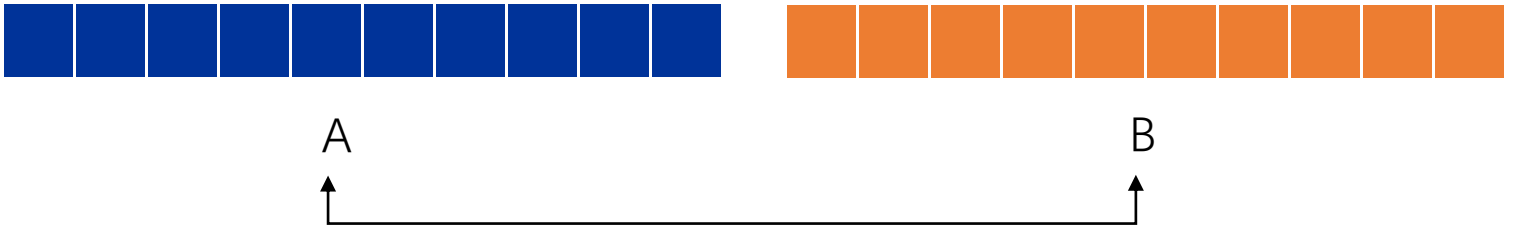
$$T(n) = 2T\left(\frac{n}{2}\right) + f(n)$$

合 归并的时候还要**计算两个子序列之间的逆序对数**

14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

解决思路二：分治法



计算 A、B 两个子序列间的逆序对的个数

$$T(n) = 2T\left(\frac{n}{2}\right) + f(n)$$

14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

$$T(n) = 2T\left(\frac{n}{2}\right) + f(n)$$

核心问题：$f(n)$ 是什么级别这个逆序对求解方法才能有效?

有效指的是低于粗暴方法的复杂度： $O(n^2)$

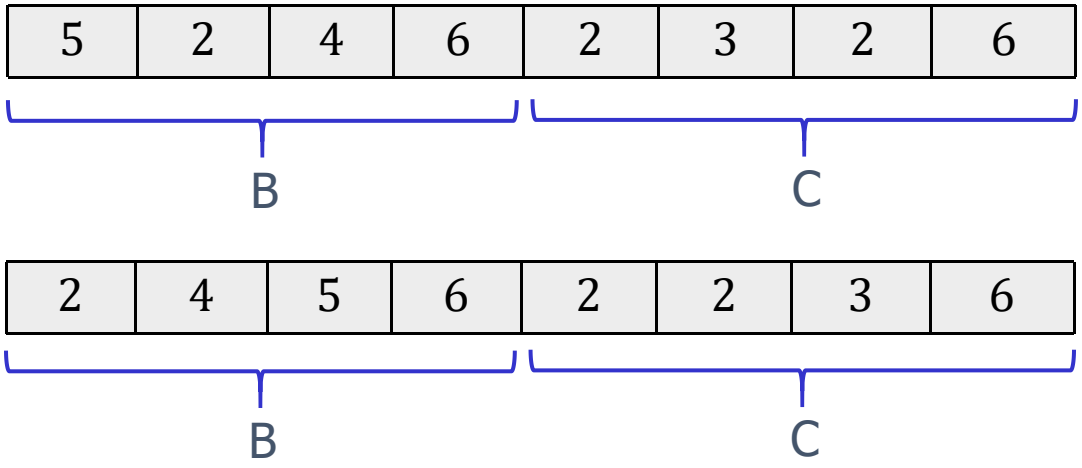
14.2.2 归并排序 – 求逆序对的个数

- 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

解决思路二：分治法 - 如果两个子序列已经排好序，它们之间的逆序数可用一个线性时间的算法算出

先将 A = {5, 2, 4, 6, 2, 3, 2, 6} 分成两个子序列：

排序后



14.2.2 归并排序 – 求逆序对的个数

□ 逆序：给定自然数 1, … , n 的一个排列，如果 j > i 但 j 排在 i 的前面则称 (j, i) 为该排列的一个逆序

- 将序列 A 分成两个子序列 B、C。首先分别计算B、C中的逆序对个数
- 然后对两个子序列进行排序，打乱了 B、C 中一部分元素的原有顺序
- 但是两个数组之间的“逆序对”的个数不会受到影响
- 所以排序过程对整个问题的求解的正确性是没有任何影响的

14.2.2 归并排序 – 求逆序对的个数

```
void MergeSort(int r[], int low, int high) {  // 进行归并排序
    int mid, r1[1000], i;
    if (low == high)
        return;
    else {
        mid = (low + high) / 2;          // 划分
        MergeSort(r, low, mid);       // 递归求左子序列中逆序对
        MergeSort(r, mid + 1, high);  // 递归求右子序列中逆序对
        Merge(r, r1, low, mid, high);    // 合并
        for (i = low; i <= high; i++)
            r[i] = r1[i];}
}
```

14.2.2 归并排序 – 求逆序对的个数

```
void Merge(int r[], int r1[], int low, int mid, int high)          // 合并子序列
{
int i = low, j = mid + 1, k = low;
while (i <= mid && j <= high) {
    if (r[i] <= r[j]) {// 取较小者放入r1[k]中
        r1[k++] = r[i++];}
    else {
        count += mid - i + 1;
        // 若左子序列中的数1大于右子序列的数2，则数1后面的数都大于数2
        r1[k++] = r[j++];}}
    while (i <= mid) r1[k++] = r[i++];
    while (j <= high) r1[k++] = r[j++];
}
```

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

□ 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题



考察重点



熟练掌握技巧

14.2.3 快速排序

参见课本：P447

- **归并排序：** 老师要求按照分数递减方式给试卷排序。两位助教各自负责一半试卷整理任务。两位助教整理好各自负责试卷后，应该如何整合所有试卷，保证所有试卷以分数递减方式排序。

Q: 两位助教是不是可以采取另一种策略?

- 随机选择一张85分试卷，A找85以上试卷，B找85以下试卷;
- A再外包C和D：随机选择一张80分试卷，C找80以上试卷，D找80以下试卷

。。。

这是快速排序

14.2.3 快速排序

参见课本：P447

- 在这种方法中，n个元素被分成三段：左段left、右段right和中段middle。
  - 中段middle仅包含一个元素；
  - 左段left中各元素都小于等于中段元素；
  - 右段right中各元素都大于等于中段元素。
- left 和 right 中的元素可以独立排序，不必对 left 和 right 的排序结果进行合并。
- middle 中的元素称为支点(pivot)。

14.2.3 快速排序（支点元素的选择）

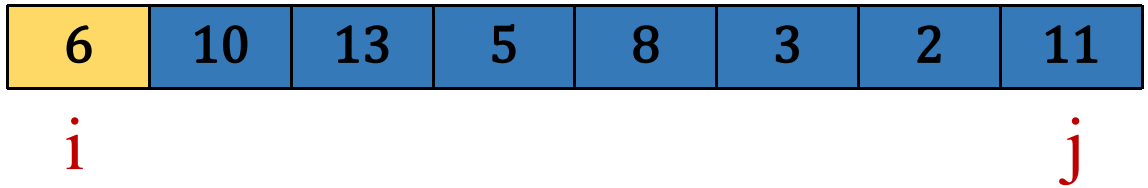
参见课本：P447

- 选择数列的第一个元素或最后一个元素。
  - 如果数列已排好序 (或倒序)，那么每一趟只确定了基准元素的位置，没有达到把数列分成左右两部分的目的，无法发挥分治法的优势。
  - 快速排序需要进行n趟，时间复杂度退化成$O(n^2)$。
- 随机选择一个元素作为支点元素。
  - 每一次有极小的概率选到数列的最大值或最小值，会影响到分治的效果。最坏时间复杂度是$O(n^2)$ 。

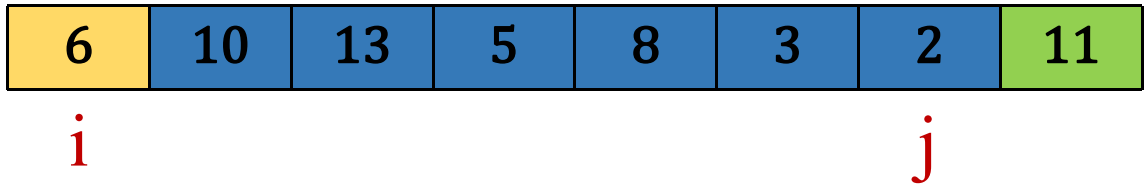
14.2.3 快速排序（元素移动）

参见课本：P447

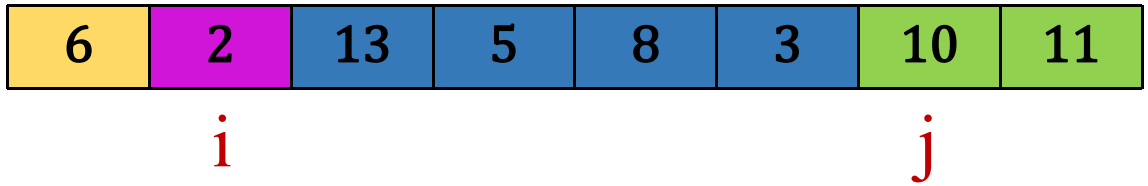
- 给数组A中8个数排序。选左边第一个数做支点。i = 0、j = 7、x = 6：



- j 向左查找。A[7] > x， A[6] < x，j 停下来：



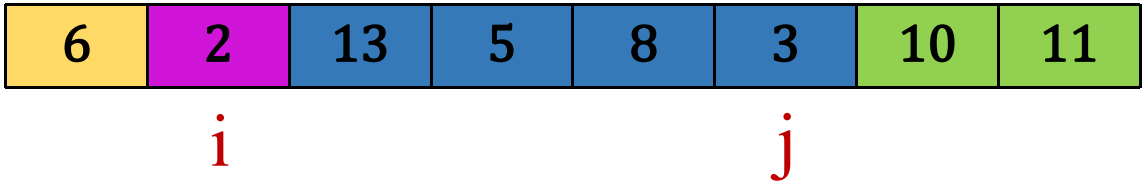
- i 向右查找。A[1] > x。A[2] ⟷A[7]：



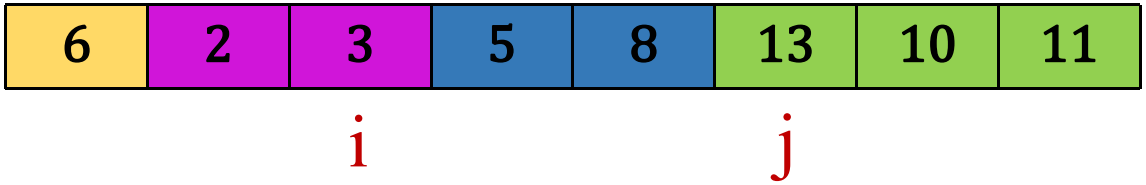
14.2.3 快速排序（元素移动）

参见课本：P447

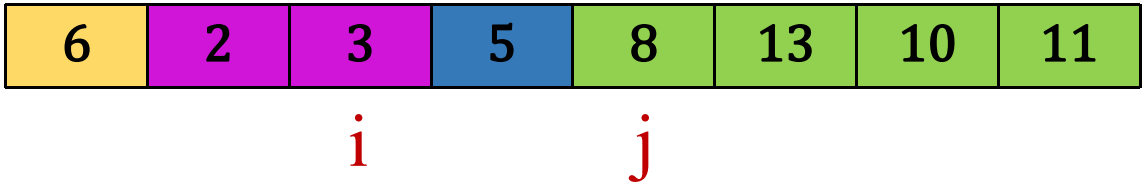
- j 向左查找。A[5] < x，j 停下来：



- i 向右查找。A[2] > x。交换 A[i] 和 A[j]：



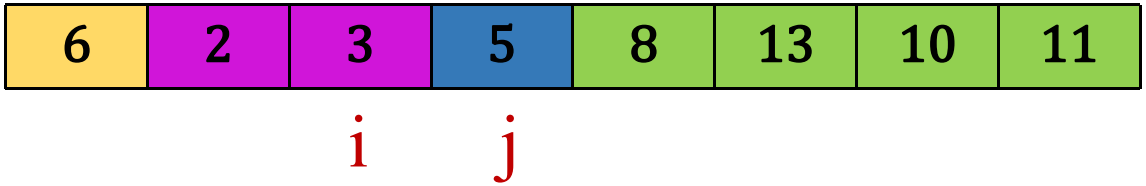
- j 向左查找：



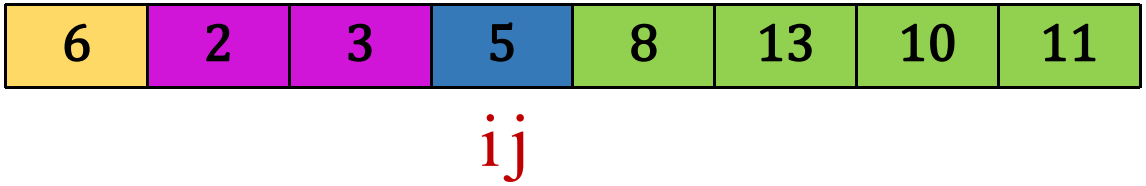
14.2.3 快速排序（元素移动）

参见课本：P447

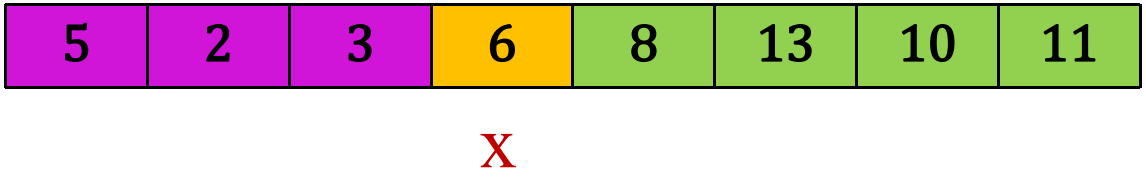
- j 向左查找， A[4] > x，A[3] < x， j停下来：



- i 向右查找。遇到了 j，搜查结束：



- 交换基准点和这个相遇点：



14.2.3 快速排序（伪代码）

参见课本：P447

```
1.  Partition (A, p, r)  //A[p..r]
2.      x ← A[p]   // pivot = A[p]
3.      i ← p+1
4.      j ← r
5.      while (A[j] > x && i < j) j —;
6.      while (A[i] < x && i < j) i ++;
7.      if  (i<j)
8.          exchange A[i] ⟷A[j]
9.      exchange A[p] ⟷A[i]
10.   return i
```

14.2.3 快速排序（伪代码）

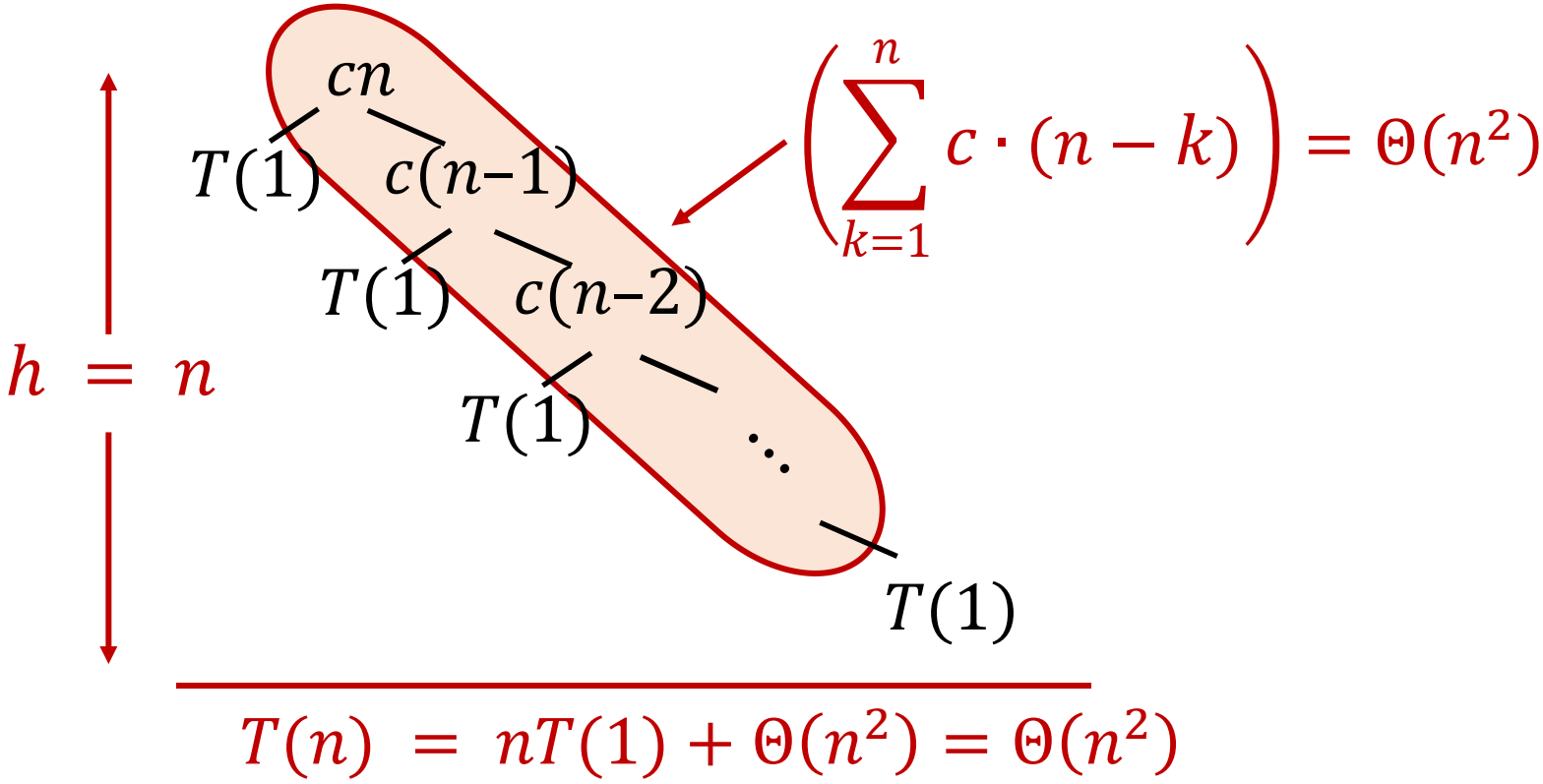
参见课本：P447

```
1.  QuickSort (A, p, r)
2.      if p<r
3.          then q ← Partition(A, p, r)
4.          QuickSort (A, p, q-1)
5.          QuickSort (A, q+1, r)
```

- Initial call: QuickSort (A, 1, n)
  - 对于输入序列 A[p..r]，算法Partition的计算时间为 $O(r-p-1)$。

14.2.3 快速排序（最坏情况递归树）

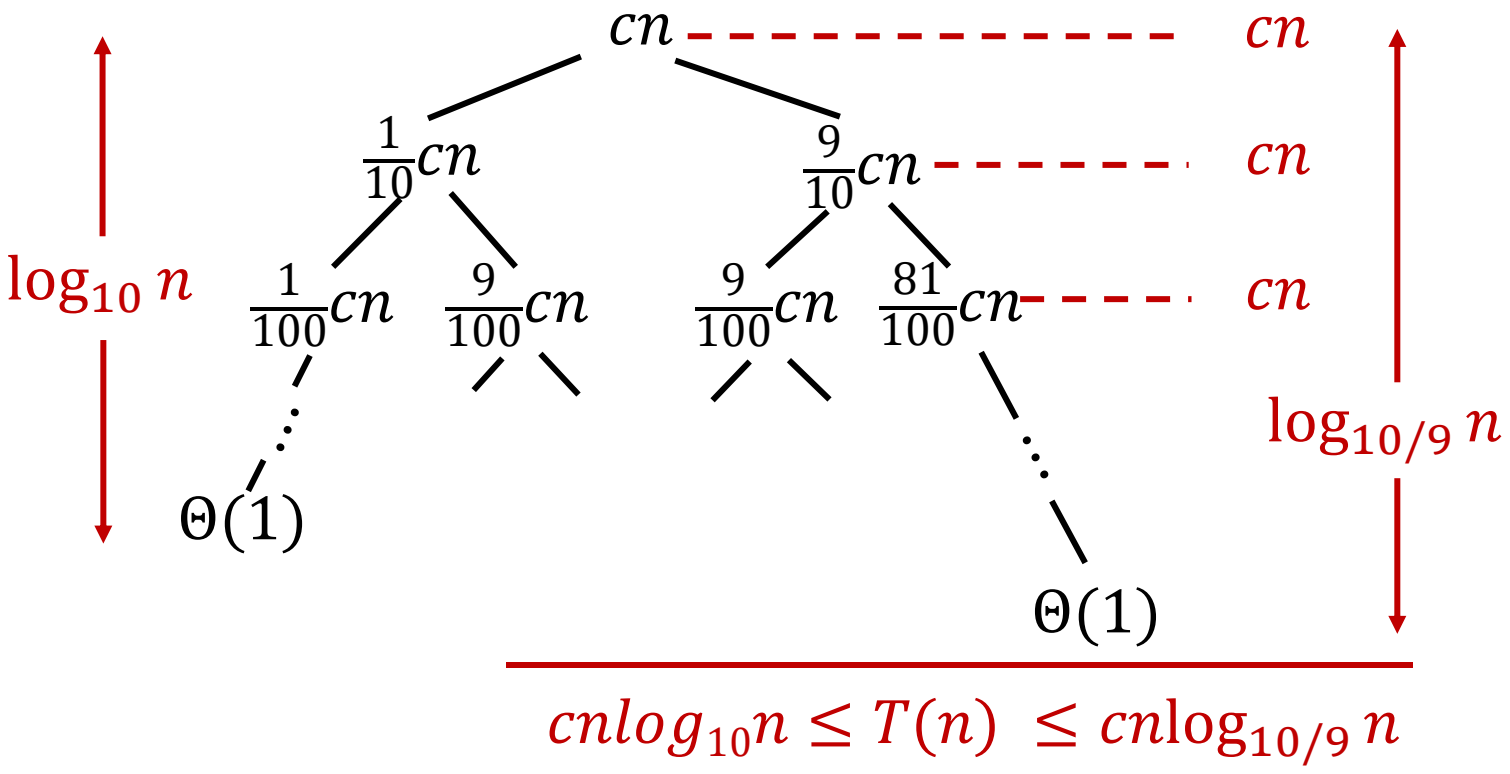
- 最坏情况: $T(n) = T(1) + T(n-1) + cn = O(n^2)$。



14.2.3 快速排序（好的情况的递归树）

参见课本：P450

- 最好情况: $T(n) = 2T(n/2) + O(n) = O(n\log n)$。



- 平均情况: $T(n) = O(n\log n)$，参见课本P449页定理14-1。

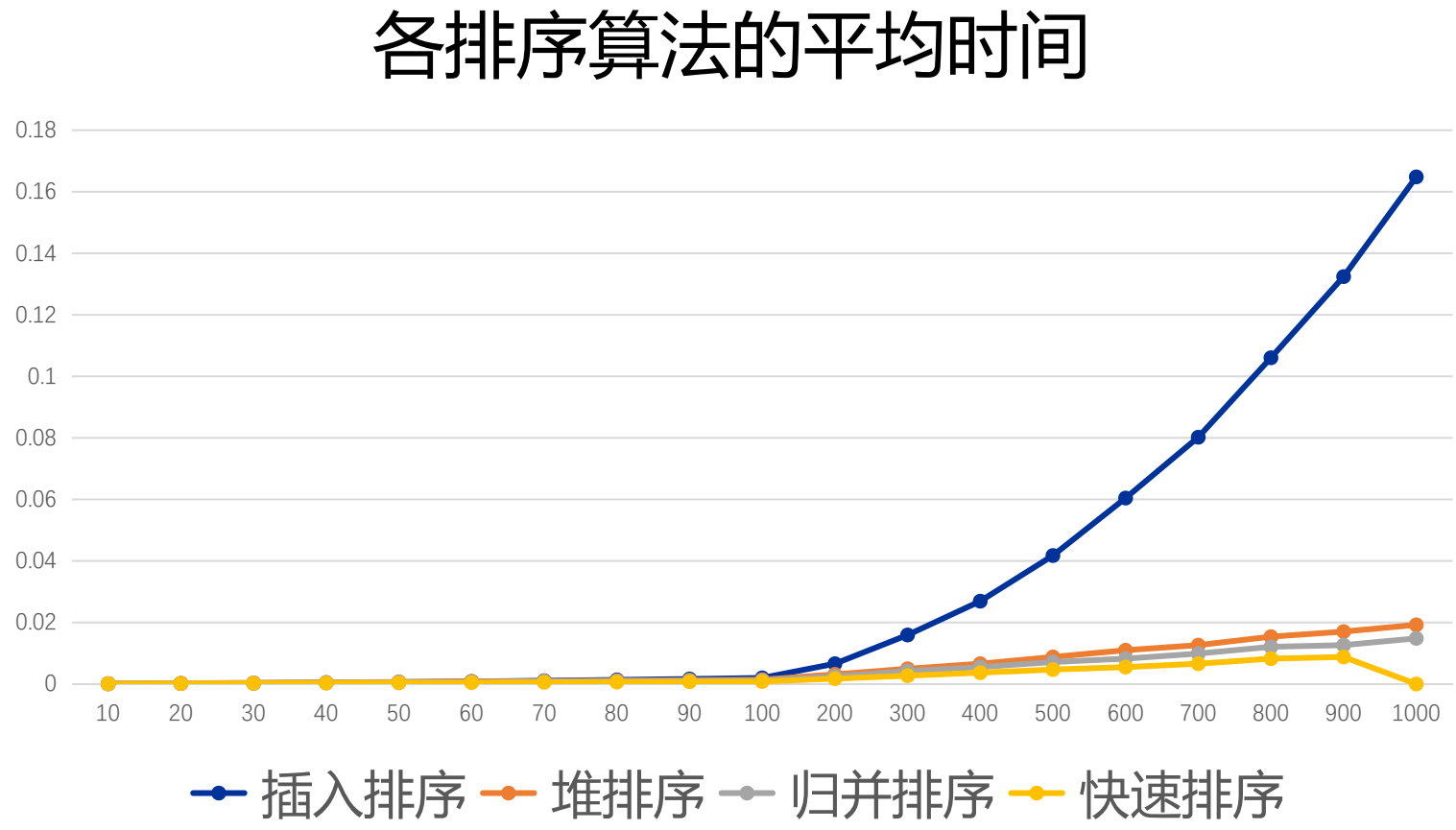
14.2.3 快速排序（排序算法比较）

参见课本：P450

方法	最坏复杂性	平均复杂性
冒泡排序	n^2	n^2
计数排序	n^2	n^2
插入排序	n^2	n^2
选择排序	n^2	n^2
堆排序	$n\log n$	$n\log n$
归并排序	$n\log n$	$n\log n$
快速排序	n^2	$n\log n$

14.2.3 快速排序（排序算法比较）

参见课本：P451



目录大纲

分治法概述

- 引例：伪币问题
- 基本思想
- 设计方法与实例
  - 金块问题
  - 矩阵相乘
  - 大整数乘法
  - 二分搜索

分治法的应用实例

- 棋盘覆盖
- 循环赛日程表
- 归并排序
- 快速排序
- 寻找第k小
- 平面点对
- 平面点集凸包问题

考察重点　　熟练掌握技巧

14.2.4 寻找第k小(选择问题)

参见课本：P452

- 问题：对于给定的 n 个元素的数组 a[0..n-1]，要求从中找出第 k 小的元素。

直接方法

首先对这 n 个元素进行排序

☞(可在O(nlogn)时间内解决)，然后选择第 k 个元素。

VS

分治法

win!

数组 a[p..q] 被划分为两个子数组 a[p.. r–1] 和 a[r+1..q]，使 a[p..r–1] 中每个元素都不大于 a[r+1..q] 中每个元素。

- 如果 k = r–p+1，支点元素就是第 k 个元素；
- 如果 k < r–p+1，下一次搜索在 a[p..r–1] 中进行；
- 反之，在 a[r+1..q] 中进行。

14.2.4 寻找第k小(选择问题)

参见课本：P452

```
RAND-SELECT(A, p, q, k)
    // k th smallest of A[p..q]
  if p = q then return A[p]
  r ← RAND-PARTITION (A, p, q)
  i ← r-p+1                    // i = rank(A[r])
  if k = i then return A[r]
  if k < i
       then return RAND-SELECT(A, p, r-1, k)
  else return RAND-SELECT(A, r+1, q, k-i)
```

➢分治法的解决思路

如果 k = r–p+1，支点元素就是第 k 个元素

如果 k < r–p+1，下一次搜索在 a[p..r–1] 中进行

反之，在 a[r+1..q] 中进行

14.2.4 寻找第k小(选择问题)

参见课本：P452

- 最坏情形为支点元素是最大或最小元素：



$$T(n) = T(n-1) + O(n) = O(n^2)$$

如何保证选出的支点不是最大或最小元素？

- 如果能找到一种支点选择策略，使得支点元素划分出的两个子数组的长度都至少为原数组长度的 ε 倍（$0 < \varepsilon < 1$），那么：

$$T(n) \le T(\varepsilon \cdot n) + cn = O(n) \text{ (主定理)}$$

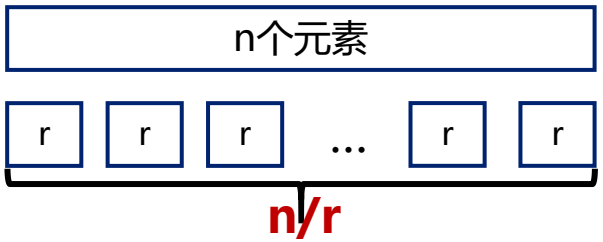
- 这样的支点选择策略下，快速排序的时间复杂度是O(nlogn)。

14.2.4 寻找第k小(选择问题)

参见课本：P453

“中间的中间”规则

- 一种选择支点元素的方法是使用“**中间的中间**（median-of-median）”规则：
  1. 将数组 a 中的 n 个元素分成 n/r 组，r 为某一整常数；
  2. 除了最后一组外，每组都有 r 个元素。
  3. 对每组中的 r 个元素进行排序，寻找每组的中位数。
  4. 对所得到的 n/r 个中位数 (中间元素)，递归使用选择算法，求得“中间之中间”作为支点元素。





对n/r个中位数递归使用选择算法选择中间数作为支点

14.2.4 寻找第k小(选择问题)

参见课本：P453

➢ r=5, n=27, A=[2, 6, 8, 1, 4, 10, 20, 6, 22, 11, 9, 8, 4, 3, 7, 8, 16, 11, 10, 8, 2, 14, 15, 1, 12, 5, 4]

1. 将该数组的27个元素分为6组，并将其排序好，考虑前5组

2. 将该数组的27个元素分为6组，并将其排序好，考虑前5组



14.2.4 寻找第k小(选择问题)

参见课本：P453

- 第一、三、四组15个元素中**至少有9个**元素**不大于中位数10**，即最少 $\lfloor \lfloor n/5 \rfloor /2 \rfloor \cdot 3$ 个元素不大于支点元素。

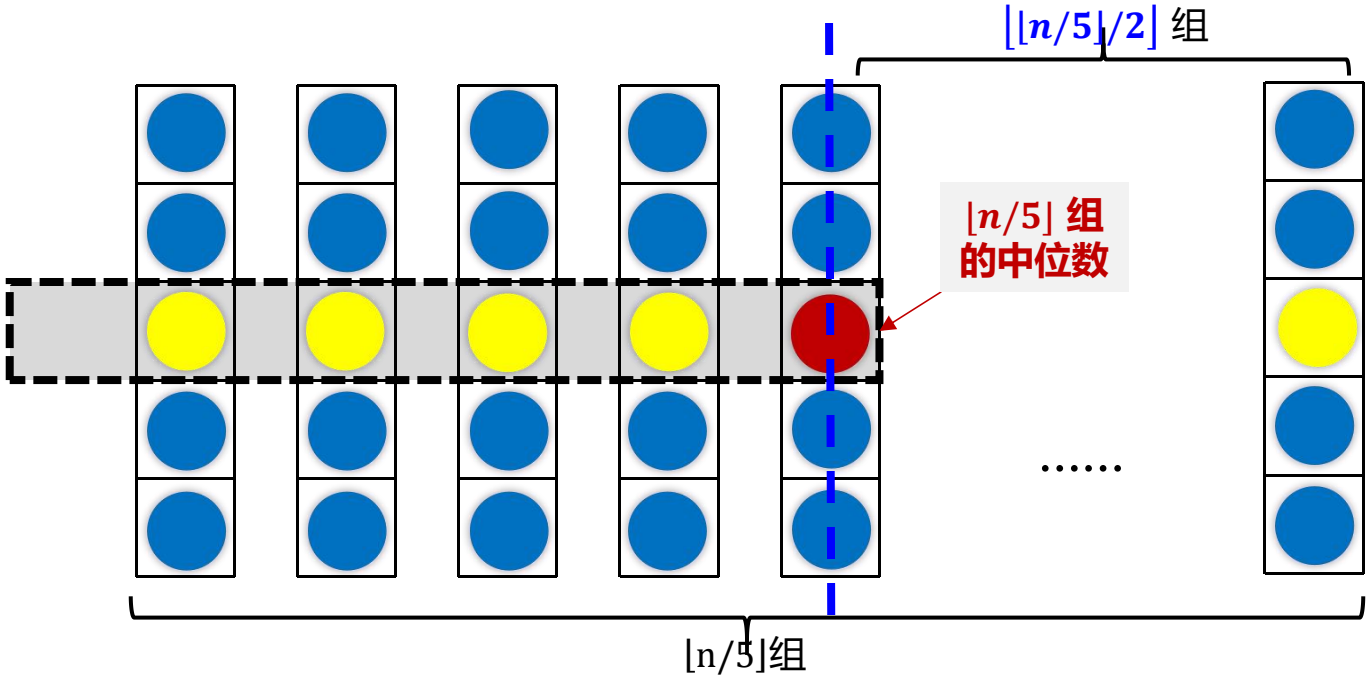
1	6	3	8	1	5
2	10	4	8	2	4
4	11	7	10	12	
6	20	8	11	14	
8	22	9	16	15	



14.2.4 寻找第k小(选择问题)

参见课本：P453

- 把 n 个**互不相同**的元素分成 $\lfloor n/5 \rfloor$ 组。
- 令 x 为中间的中间，至少有 $\lfloor \lfloor n/5 \rfloor /2 \rfloor = \lfloor n/10 \rfloor$ 组的中位数 ≤ x



14.2.4 寻找第k小(选择问题)

参见课本：P453

- **结论：**
  - 至少有 $3 \times \lfloor n/10 \rfloor$ 个元素 ≤ (或 ≥) x
  - 至多有 $n - \lfloor \lfloor n/10 \rfloor \times 3 \rfloor$ 个元素 ≥ (或 ≤) x.

对于一般分组元素为r个的时候

至少有 $\frac{(r+1)}{2} \times \lfloor n/2r \rfloor$ **个元素大于或小于x**

14.2.4 寻找第k小(选择问题)

参见课本：P453

1. 将 n 个元素分成 5 个一组。找出每组中 5 个元素的中位数。 $\Theta(n)$
2. 递归地选择 ⌊ n/5⌋ 组中位数的中位数作为支点。 $T(n/5)$
3. 利用支点把元素分成两组，i 为 x 的序号。 $\Theta(n)$
4. if k = i then return x
5. elseif k < i
6. then
7. 在左子数组里递归地选择第 k 小元素
8. else
9. 在右子数组里递归地选择第 k – i 小元素

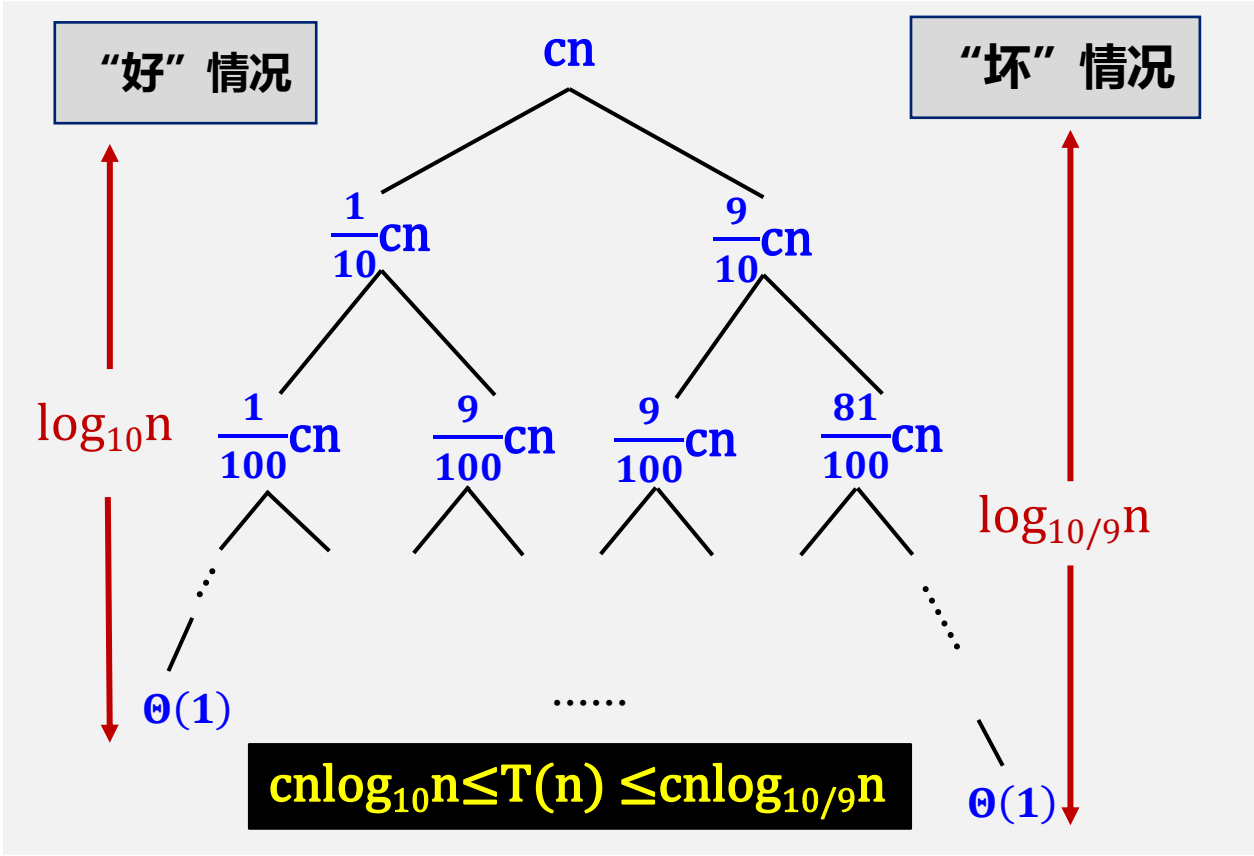
$T(\alpha n)$

在n个元素中找中位数的时间复杂度为 $\Theta(n)$

$$T(n) \leq T(n/5) + T(\alpha n) + \Theta(n)$$

14.2.4 寻找第k小（递归树）

□ 最好情况: T(n) = 2T(n/2)+O(n)=O(nlogn)



❶ 左右两个树中**选一个**，不是两个都选

❷ 假设每次选的都是最糟糕的情况，即$\frac{9}{10}$衰减那支，此时对应**最差情况复杂度**

寻找**T(αn) 的最大值**

14.2.4 寻找第k小（r=5）

■**定理18-3.1** 若 r = 5，且 A 中所有元素都不同，那么当 n ≥ 24 时，有max{| left |, | right | } ≤ 3n/ 4。

表示在计算最差的情况的时候一个分支下降的有多慢

■**证明：**

由前面的分析知，大于支点元素 x 的元素数目有上界

$$n - \lfloor \lfloor n/5 \rfloor / 2 \rfloor \cdot 3 \le n - 3 \cdot (n-4)/10 = 0.7n + 1.2$$

当 n ≥ 24 时，**0.7n + 1.2 ≤ 3n/4**。相应的，小于支点元素 x 的元素个数也有相同的上界。

证毕

14.2.4 寻找第k小（r=5）

- 这意味着无论第 k 小元素落于x 左侧还是右侧，时间复杂度函数都有上界：

$$T(n) \le T(n/5) + T(3n/4) + cn$$

- 用归纳法可以证明，当 $n \ge 24$ 时，有 $T(n) \le 20cn$。

结论：利用中间的中间做支点，可以在**线性时间**内找到第 k 小元素

14.2.4 寻找第k小（r=7）

- **定理18-3.2** **若 r = 7，且 A 中所有元素都不同，那么当 n ≥ 15 时，有max{| left |, | right | } ≤ 5n/6。**
- **证明** 类似于 r = 5 的情形，当 r = 7 时，大于支点元素的元素个数有上界

$$n - \left\lfloor \lfloor n/7 \rfloor / 2 \right\rfloor \cdot 4 \le n - 2 \cdot (n-6)/7$$

当 n ≥ 15 时，n – 2 · (n – 6)/7 ≤ 5n/6。　　证毕

结论： 当 n ≥ 15 时，可以用归纳法证明 T(n) ≤ T(n/7) + T(5n/6) + cn ≤ 42cn。可以在**线性时间**内找到第 k 小。

14.2.4 寻找第k小（r=3）

■**定理18-3.4** **若 r = 3，找不到 α ∈ (1/2, 2/3) 使得 max{| left |, | right | } ≤ αn。**

■**证明** 当 r = 3 时，至少有$\lfloor \frac{n}{3} \rfloor$个元素 ≤ x，至多有 n – $\lfloor \frac{n}{3} \rfloor$ 个元素 ≥ x。

令 $n - \lfloor \frac{n}{3} \rfloor \le n - \frac{n-2}{3} \le \alpha n$，得 **(α – 2/3)n ≥ 2/3**。找不到满足 α + 1/3 < 1 的 α 使上式成立。

结论： 当 r = 3 时，算法**不能保证**在 O(n) 时间内求解。

14.2.4 寻找第k小（时间复杂度）

算法的时间复杂度满足方程：

$$W(n) = W(n/3) + W(4n/6) + cn$$

由递归树可得：

$$W(n) = \Theta(n\log n)$$



关键： |M|与归约后子问题规模之和小于n，

递归树每行的工作量构成公比小于1的等比

级数，算法复杂度才是$\Theta(n)$

目录大纲

口 分治法概述

- 引例：伪币问题
- 基本思想



设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

口 分治法的应用实例



棋盘覆盖

- 循环赛日程表



归并排序



快速排序



寻找第k小

- 平面点对
- 平面点集凸包问题

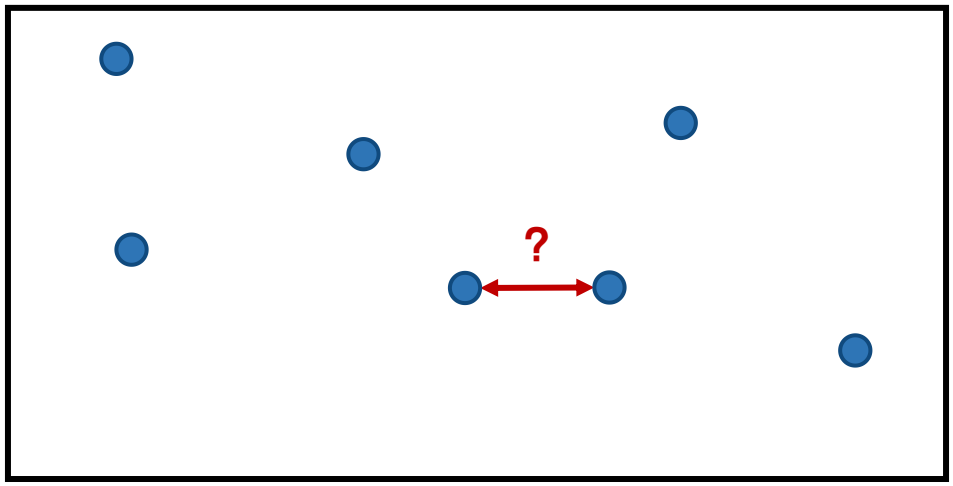
 **考察重点**

 **熟练掌握技巧**

14.2.5 最接近点对问题

参见课本：P454

- **例13-2**：给定平面上 n 个点的集合 S，找其中的一对点，使得在 n个点组成的所有点对中，该点对间的距离最小。严格来讲，最接近点对可能多于一对，为简便起见，我们只找其中的一对作为问题的解。

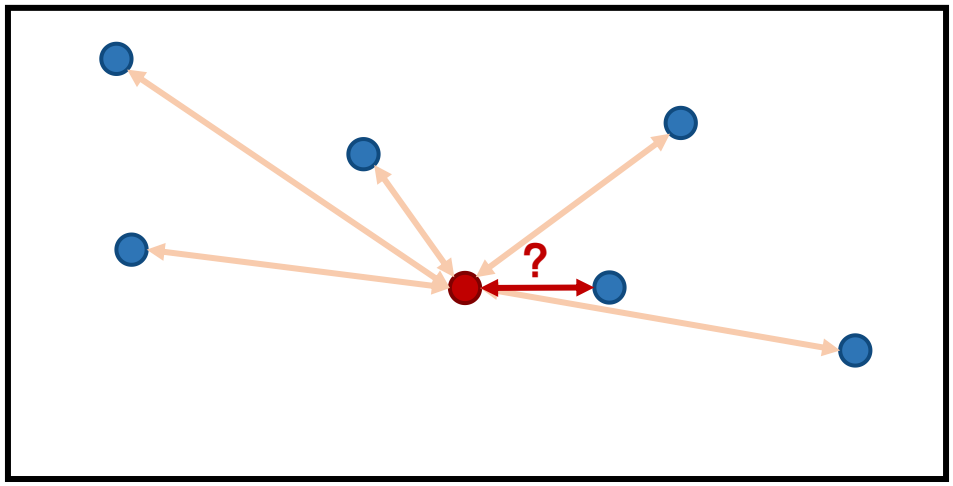


哪两个点距离最小?

14.2.5 最接近点对问题

参见课本：P454

- **考虑穷举法：**一个简单的方法是将每个点与其他 n-1 个点的距离算出，找出**最小距离**即可。
- 时间复杂度T(n)=n(n-1)/2+n=**O(n²)**

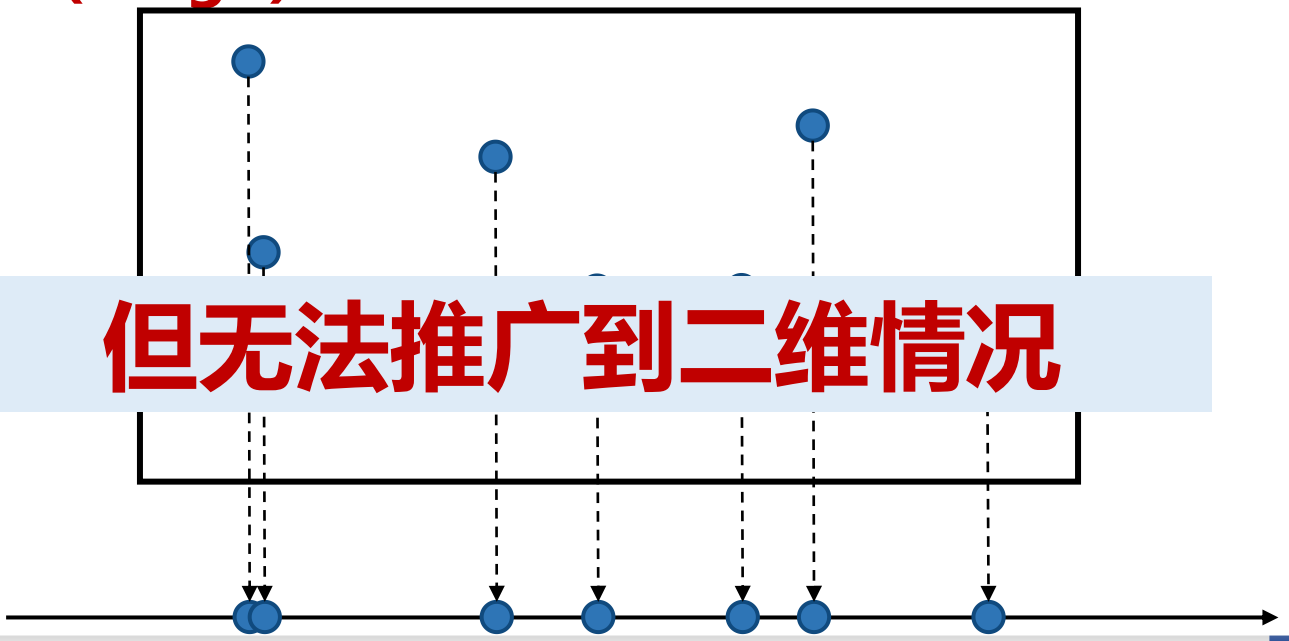


哪两个点距离最小?

14.2.5 最接近点对问题

参见课本：P454

- **一维情况**：将集合S中的n个点退化为x轴上的n个实数 $x_1,x_2,x_3,\ldots,x_n$，把它们排好序，再进行一次线性扫描即可找出。这样最接近点对是这n个实数中相差最小的2个。此时 T(n)=**O(nlogn)**

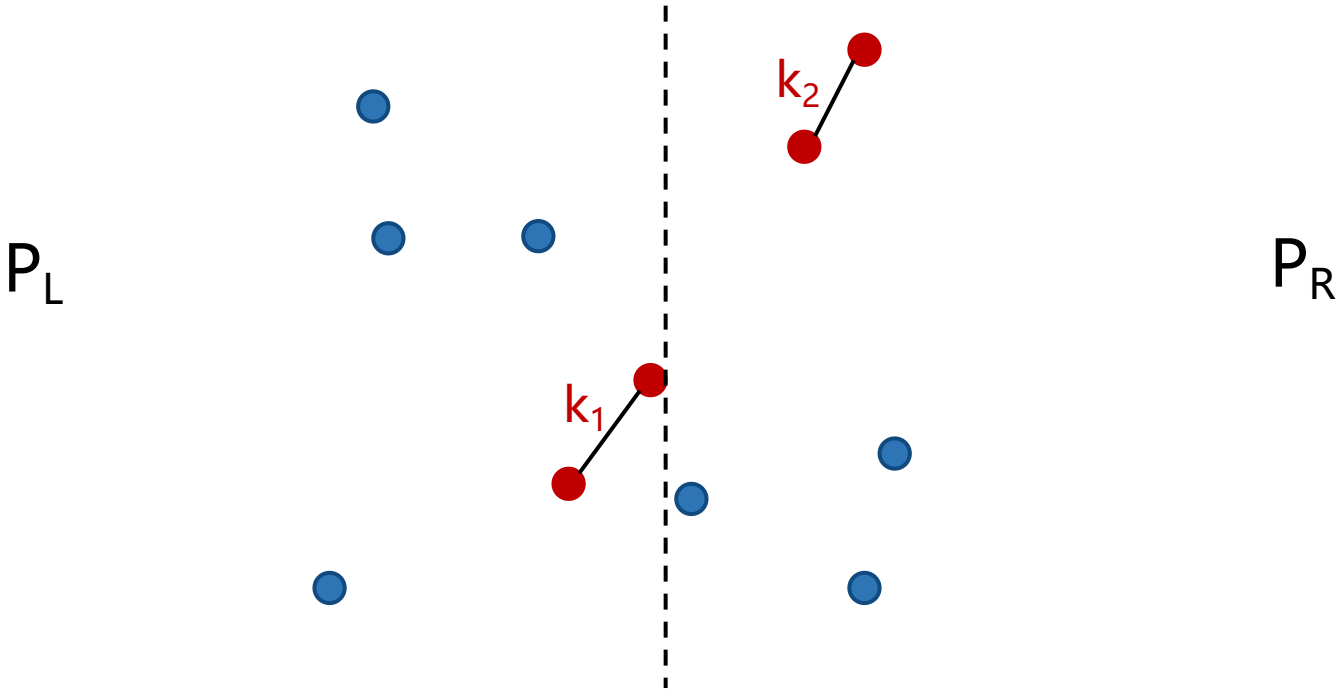


但无法推广到二维情况

14.2.5 最接近点对问题

参见课本：P454

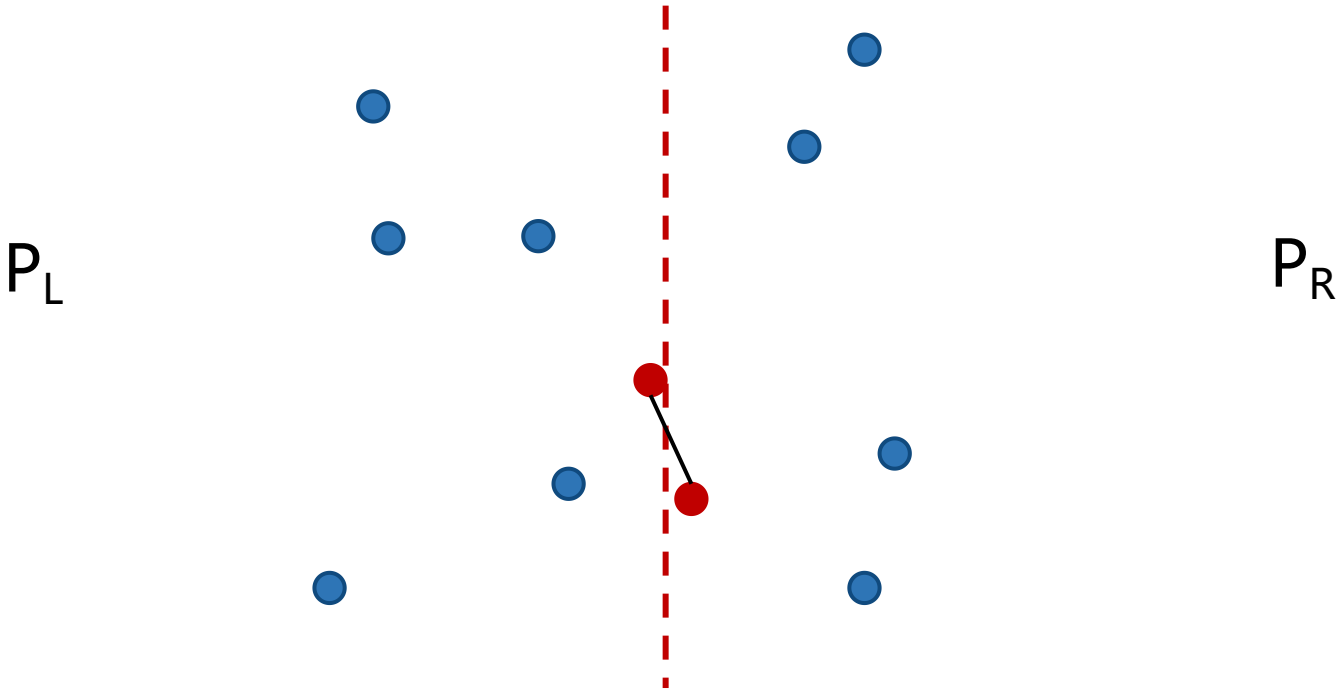
- **分治策略**：能否找到一种时间复杂度更低的方法？
  - **分(Divide)**：将点集P依据横坐标分为左右两个子集P_L和P_R
  - **治(Conquer)**：分别计算P_L和P_R两个点集的最近点对k_1，k_2



14.2.5 最接近点对问题

参见课本：P454

- **分治策略**：能否找到一种时间复杂度更低的方法？
  - **合(Combine)**：考虑P_L和P_R两个点集**交界处**的最近点对，因为交界处可能有比k_1和k_2距离更近的两个点



14.2.5 最接近点对问题

参见课本：P454

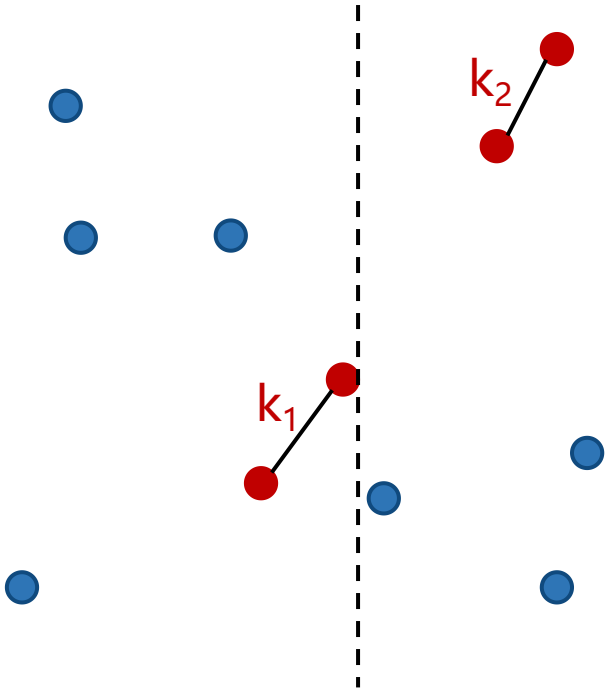
口 算法伪码：

MinDistance(P, X, Y)

输入: 点集P, X和Y为横、纵坐标数组

输出: 最近的两个点及距离

1. 若|P|≤3，直接计算其最小距离
2. 排序X, Y
3. 做中垂线l将P划分为P_L和P_R
4. MinDidtance (P_L,X_L,Y_L)
5. MinDistance (P_R,X_R,Y_R)
6. $\delta=\min(\delta_L,\delta_R)$//$\delta_L$, δ_R为子问题的距离
7. 检查距中垂线l不超过δ两侧各1个点的距离，若小于δ，修改δ为这个值



14.2.5 最接近点对问题

参见课本：P454

口 算法分析：

步1.递归边界处理：　　*O(1)*

步2.排序：　　*O(nlogn)*

步3.划分：　　*O(1)*

步4-5.子问题　　*2T(n/2)*

步6.确定δ：　　*O(1)*

步7.检查跨边界点对：　　*O(?)*

$$T(n)=\begin{cases}1, & n=2\\ 3, & n=3\\ 2T(n/2)+O(\mathrm{f}(n)), & n>3\end{cases}$$

MinDistance(P, X, Y

输入: 点集P, X和Y为横、纵坐标数组

输出: 最近的两个点及距离

1. 若|P|≤3，直接计算其最小距离
2. 排序X, Y
3. 做中垂线l将P划分为P_L和P_R
4. MinDidtance (P_L,X_L,Y_L)
5. MinDistance (P_R,X_R,Y_R)
6. δ=min(δ_L,δ_R)//δ_L, δ_R为子问题的距离
7. 检查距中垂线l不超过δ两侧各1个点的距离，若小于δ，修改δ为这个值

f(n)是问号?和nlogn中更高的阶，表示分解和合并子问题的成本

14.2.5 最接近点对问题

参见课本：P454

口 分析f(n)的不同情况：

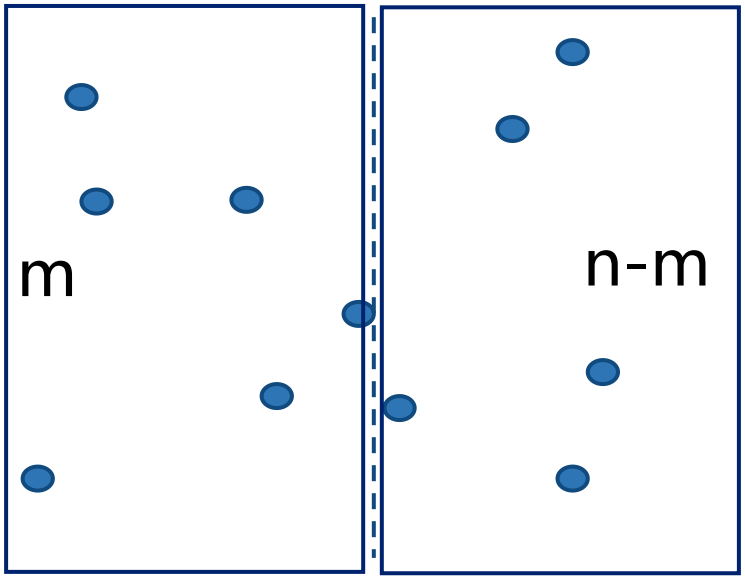
$$T(n)=\begin{cases}1, & n=2\\ 3, & n=3\\ 2T(n/2)+O(\mathrm{f}(n)), & n>3\end{cases}$$

- 若$f(n)=O(n^2)$：　　$T(n)=O(n^2)$
- 若$f(n)=O(n\log n)$：$T(n)=O(n(\log n)^2)=O(n\log^2 n)$
- 若$f(n)=O(n)$：　　$T(n)=O(n\log n)$

14.2.5 最接近点对问题

参见课本：P454

- **检查跨边界对问题**：点集总共有n个点，P_L有**m**个，则P_R有**n-m**个。如果用暴力法求左边每个点和右边每个点的距离，则需要m(n-m)次，若平分n个点(即m=n/2)，复杂度$T(n)=n^2/4$，即$O(n^2)$。

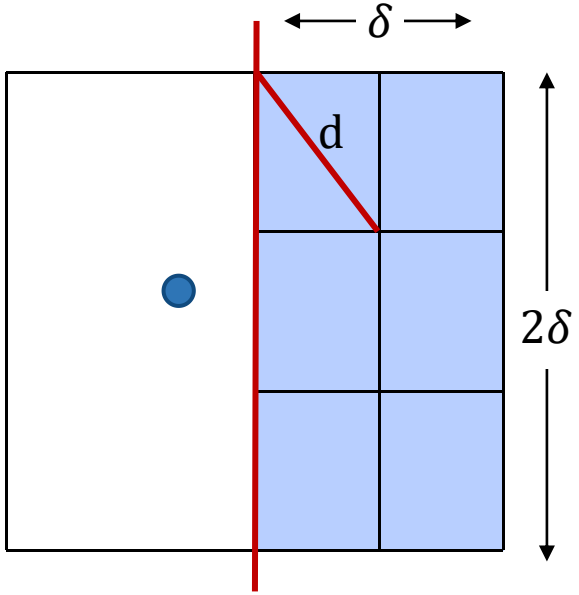


14.2.5 最接近点对问题

参见课本：P454

- **缩小边界法：**假设前面求解的最小距离为δ，那么对于左边的点，只需要在以它为中心δ*(2δ)的正方形内取值。

$$
\begin{aligned}
d &= \sqrt{(\delta/2)^2 + (2\delta/3)^2} \\
&= \sqrt{\delta^2/4 + 4\delta^2/9} \\
&= \sqrt{25\delta^2/36} \\
&= 5\delta/6
\end{aligned}
$$



由于两个点最小距离为δ，因此右边每个小方格中至多包含1个点，每个点至多需要与对面的6个点比较距离。这种比较是限行时间，因此复杂度为**O(n)**

14.2.5 最接近点对问题

参见课本：P454

□ 算法分析：

步1.递归边界处理：　　*O(1)*

步2.排序：　　*O(nlogn)*

步3.划分：　　*O(1)*

步4-5.子问题　　*2T(n/2)*

步6.确定δ：　　*O(1)*

步7.检查跨边界点对：　　*O(n)*

递归树求解

$T(n)=O(n\log^2 n)$

14.2.5 最接近点对问题

口 算法分析：

步1.递归边界处理：　*O(1)*

步2.排序：　*O(nlogn)*

步3.划分：　*O(1)*

步4-5.子问题　*2T(n/2)*

步6.确定δ：　*O(1)*

步7.检查跨边界点对：　*O(n)*

如何进一步降低这一步的复杂度？(目前排序在递归算法里)

递归树求解

$T(n)=O(n\log^2 n)$

14.2.5 最接近点对问题

- **算法分析：**以空间换时间，先将点集中的元素以它们的纵坐标进行一遍排序，然后存储在数组中，在合的步骤中以O(n)的复杂度提取并比较

递归树求解

$T(n)=O(n\log^2 n)$

$$T(n)=\begin{cases}1, & n=2\\ 3, & n=3\\ 2T(n/2)+\Theta(n), & n>3\end{cases}$$

目录大纲

□ 分治法概述

- 引例：伪币问题
- 基本思想
-  设计方法与实例
  - 金块问题
  - 矩阵相乘
  - 大整数乘法
  - 二分搜索

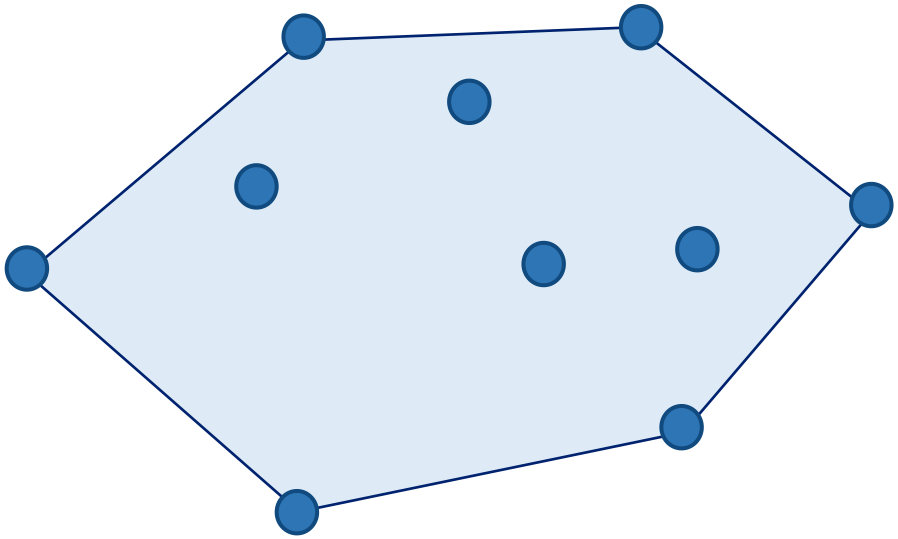
 **考察重点**  **熟练掌握技巧**

□ 分治法的应用实例

-  棋盘覆盖
  - 循环赛日程表
-  归并排序
-  快速排序
-  寻找第k小
  - 平面点对
  - 平面点集凸包问题

平面点集凸包问题

- **问题**：给定大量离散点的集合Q，求一个**最小的凸多边形**，使得Q中的点都能够处于该多边形内、或者边上。

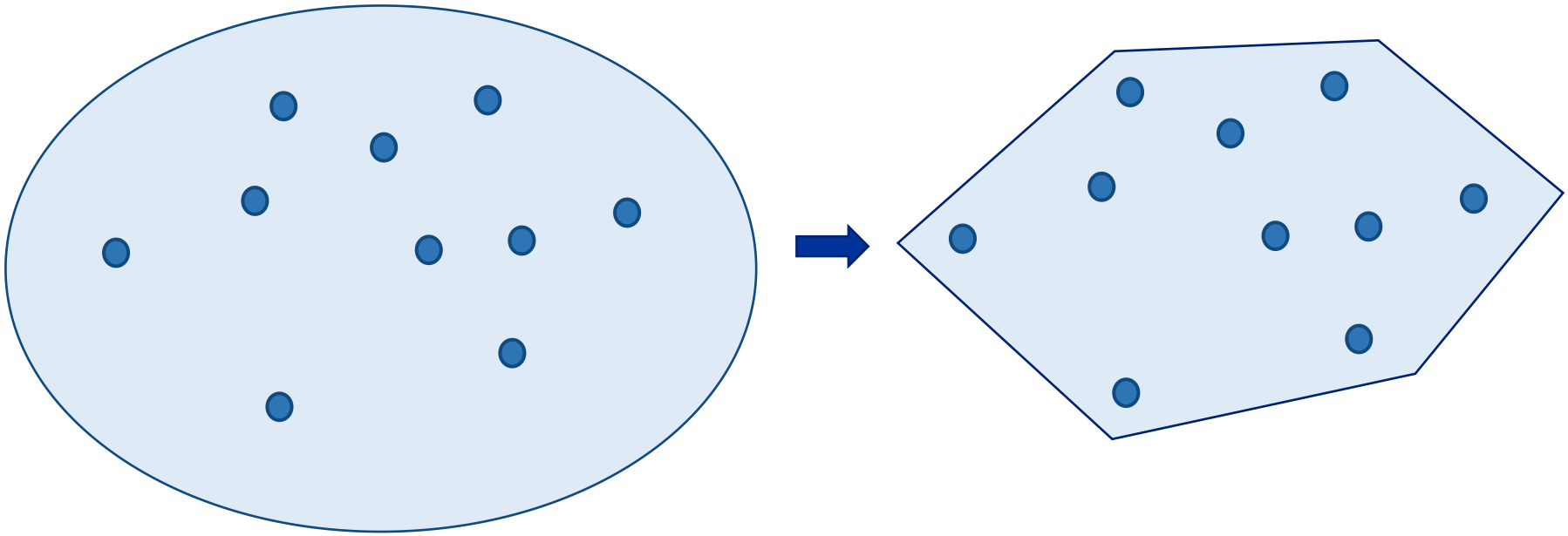


- **应用场景**

图形处理中的形状识别：字形识别、碰撞检测等

平面点集凸包问题

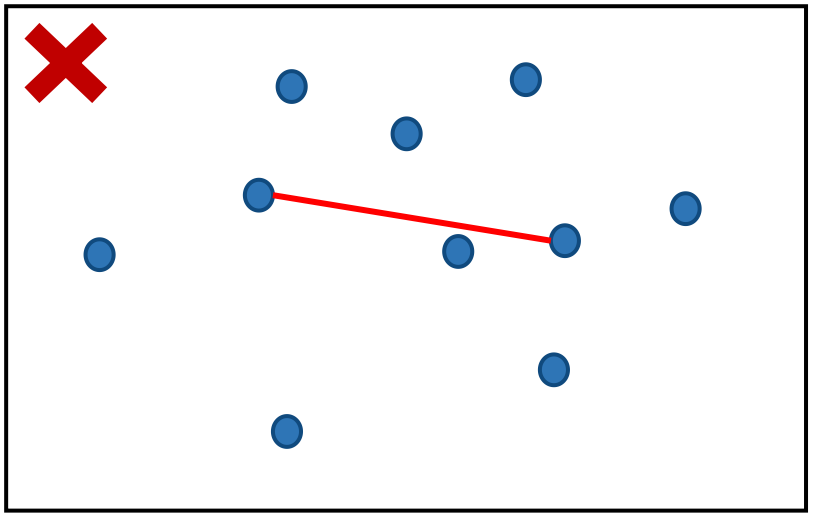
- **一种天真的思路**：用一个大圈全部包围，然后不断缩小。

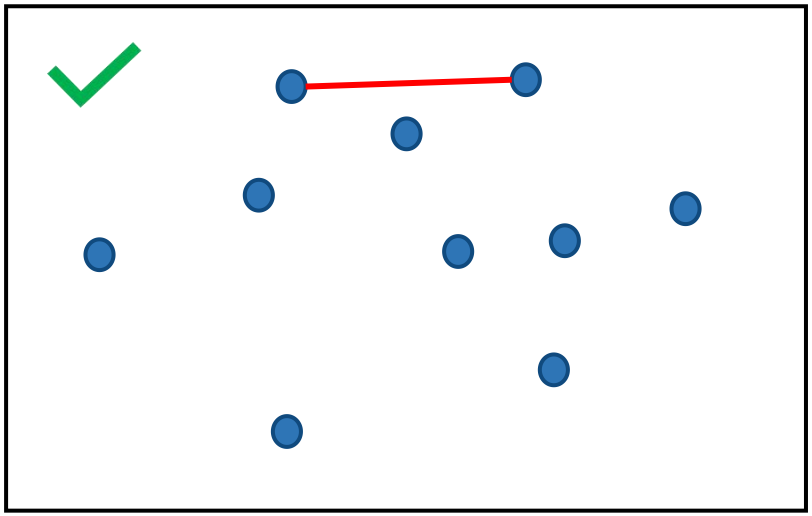


但难以在程序中模拟此过程

平面点集凸包问题

- **另一种暴力的思路：**找两点连线，如果其他点都在该连线的一侧，则这两点是凸包上的点，否则就不是。



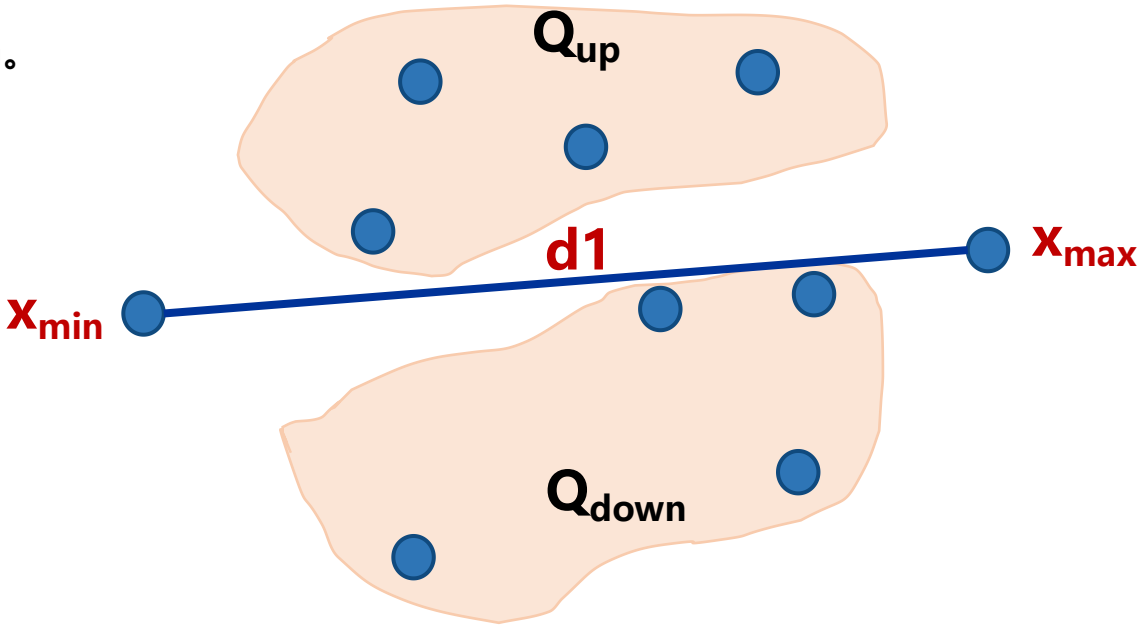


但穷举法复杂度较高。

如何用分治法去实现呢?

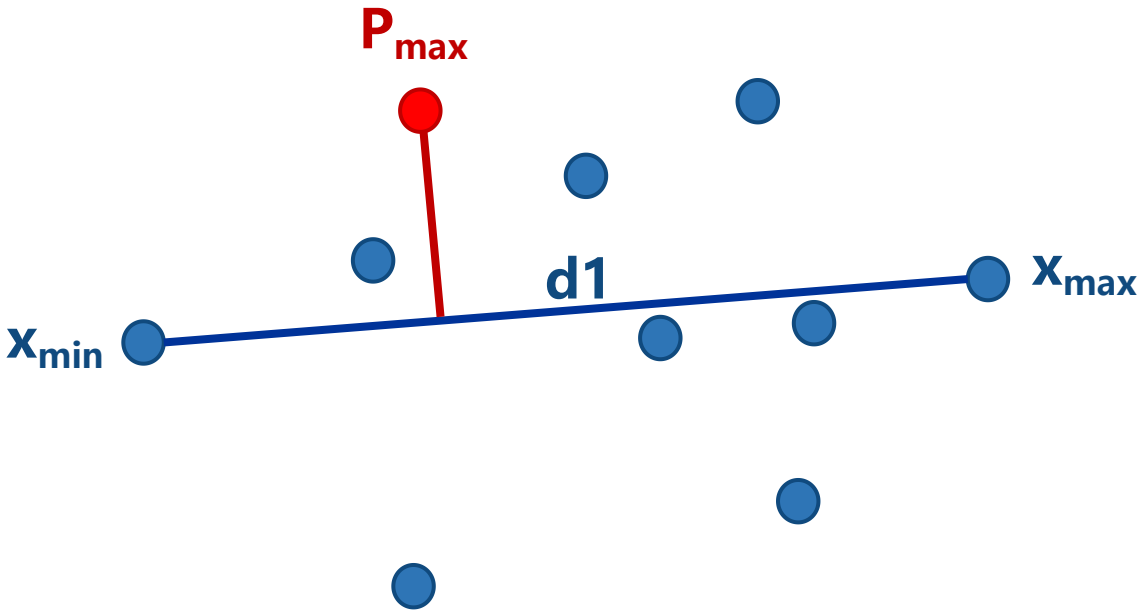
平面点集凸包问题：分治法

- **第一步：对点集内点进行排序，**连接最大横坐标点x_{max}和最小纵坐标点x_{min}，记该连线为d1，并以d为分界线将点集划分上下两部分，分别称之为上包Q_{up}和下包Q_{down}。



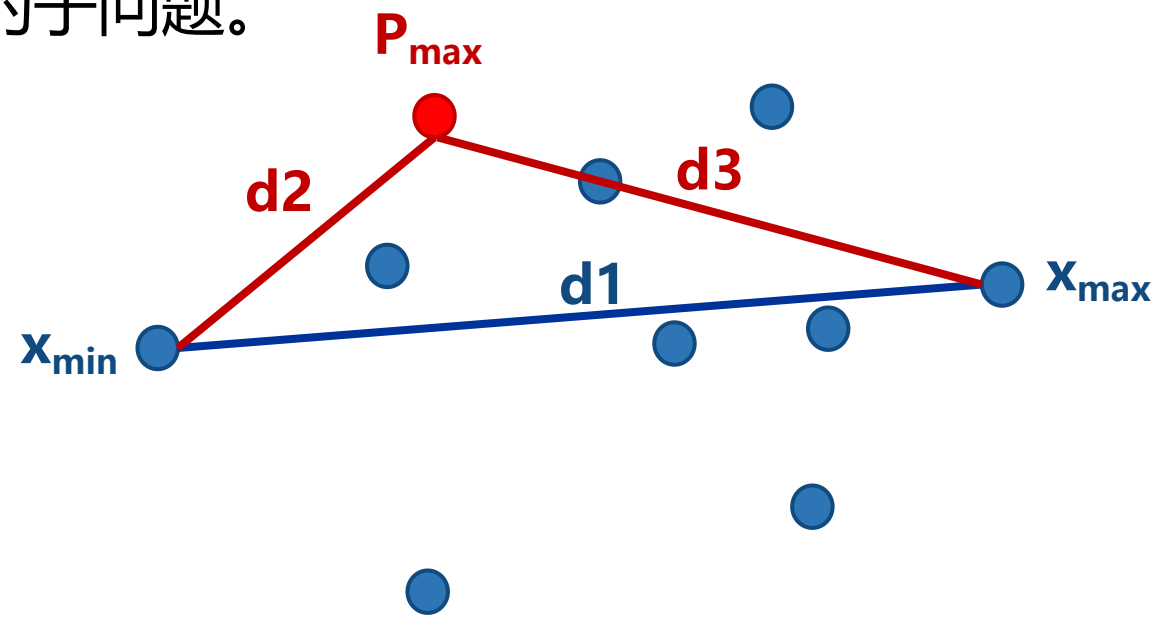
平面点集凸包问题：分治法

- **第二步**：先以上子集为例：从Q_{up}中找到一点P_{max}，该点满足到直线d1距离最远，该步骤可以由**点到直线距离公式**求解。



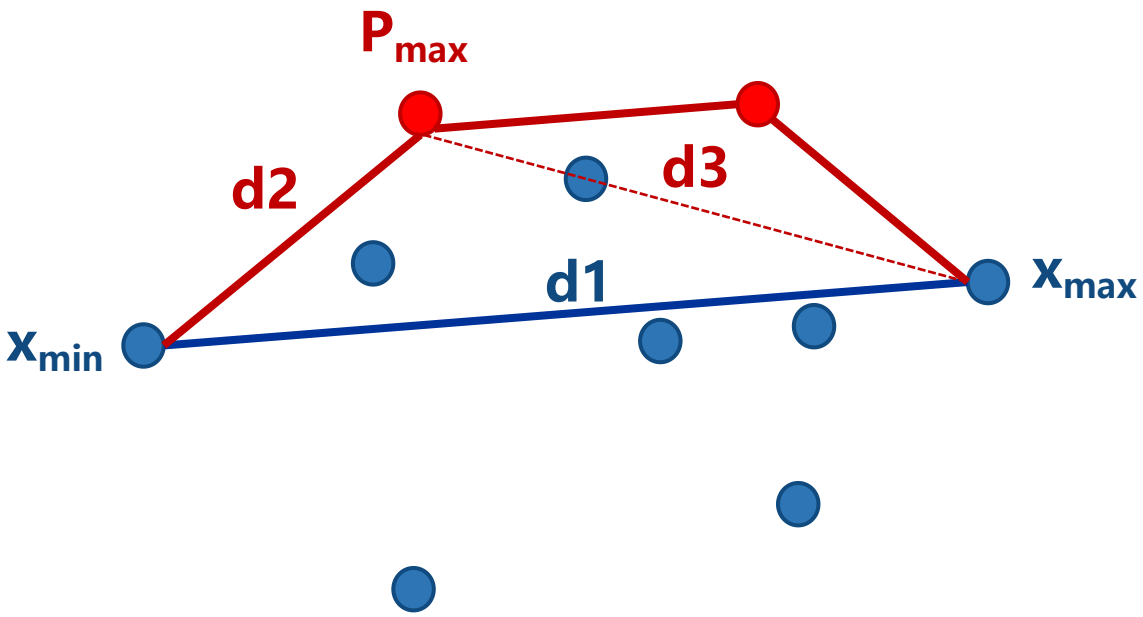
平面点集凸包问题：分治法

- **第三步：**分别连接P_{max} y_{max}、 P_{max} y_{min}，将连线分别记为d2、d3。将d2、d3外侧的点也分别看作“上包”，d2外的点与d2构成Q_{up}的子问题， d3外的点与d3也构成Q_{up}的子问题。



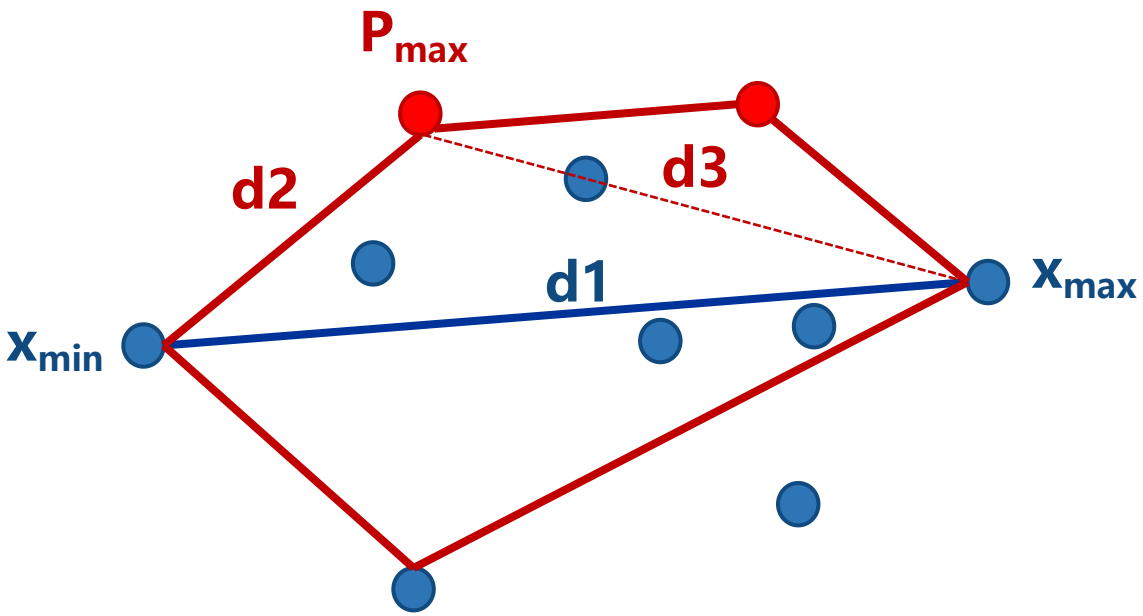
平面点集凸包问题：分治法

- **第四步**：重复前几步，对下包Q_{down}也进行同样操作。



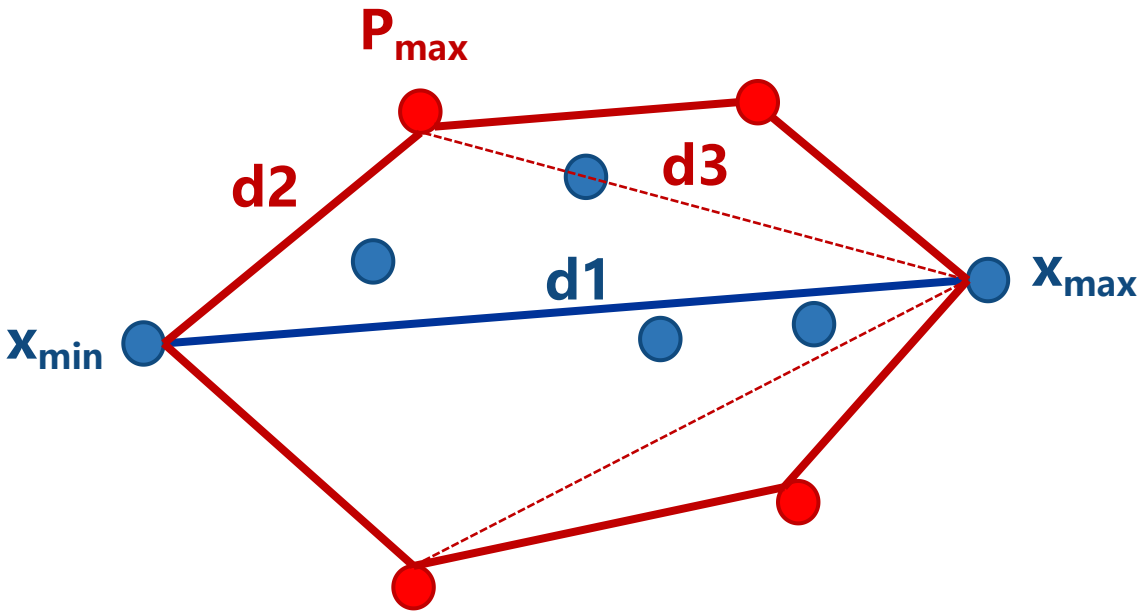
平面点集凸包问题：分治法

- **第四步**：重复前几步，对下包Q_{down}也进行同样操作。



平面点集凸包问题：分治法

- **第四步**：重复前几步，对下包Q_{down}也进行同样操作。最终构成满足条件的凸多边形。



平面点集凸包问题：分治法

口 算法分析

- Deal(Q_{up})：
  - ① 找出距离d1最远的点P_{max}，构成三角形，将三角形两条连线记为d2、d3
  - ② 检查Q_{up}中其他点是否在三角形内，在则从Q中删除，否则划分到d2、d3子问题中
  - ③ Deal(d2)
  - ④ Deal(d3)

平面点集凸包问题：分治法

□ 算法分析

初始对点集排序，然后用d1划分为上下包	O(n)
Deal递归调用	W(n)
- 找最远点（凸包顶点）P_{max}	O(n)
- 根据其他点的位置划分子问题	O(n)
$W(n) = W(n-1)+O(n)$ $W(3) = O(1)$	
最坏情况为$T(n) = O(n)+W(n)= O(n^2)$	$O(n^2)$

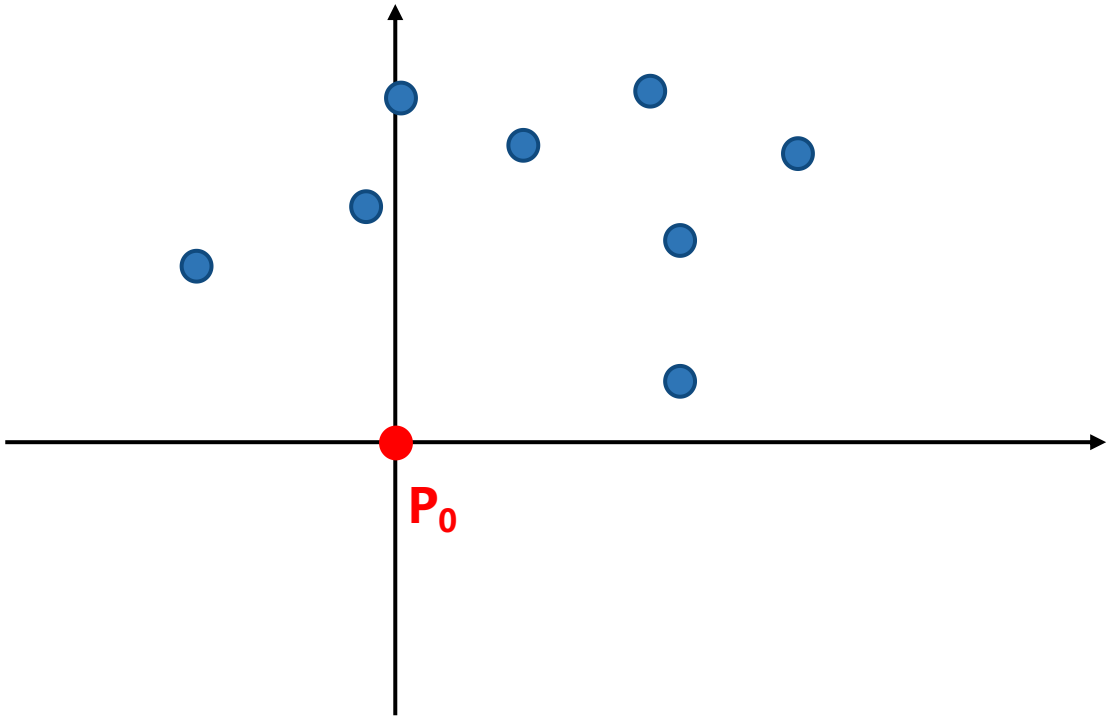
Graham扫描算法　　O(nlogn)

平面点集凸包问题：Graham扫描法

- **Graham(葛立恒)扫描法：**Graham扫描的思想是先找到凸包上的一个点，然后从那个点开始按逆时针方向逐个找凸包上的点。

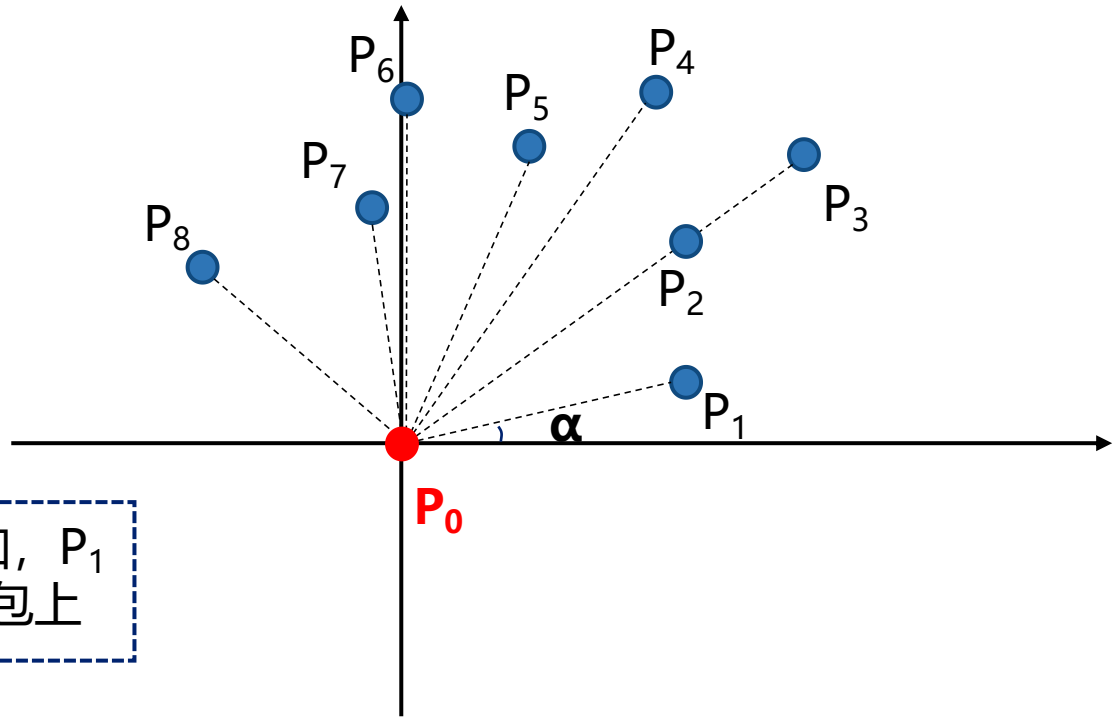
平面点集凸包问题：Graham扫描法

- **第一步**：将所有点放在二维坐标系中，纵坐标最小的点一定是凸包上的点(如P_0)。将所有点平移，使P_0作为原点。



平面点集凸包问题：Graham扫描法

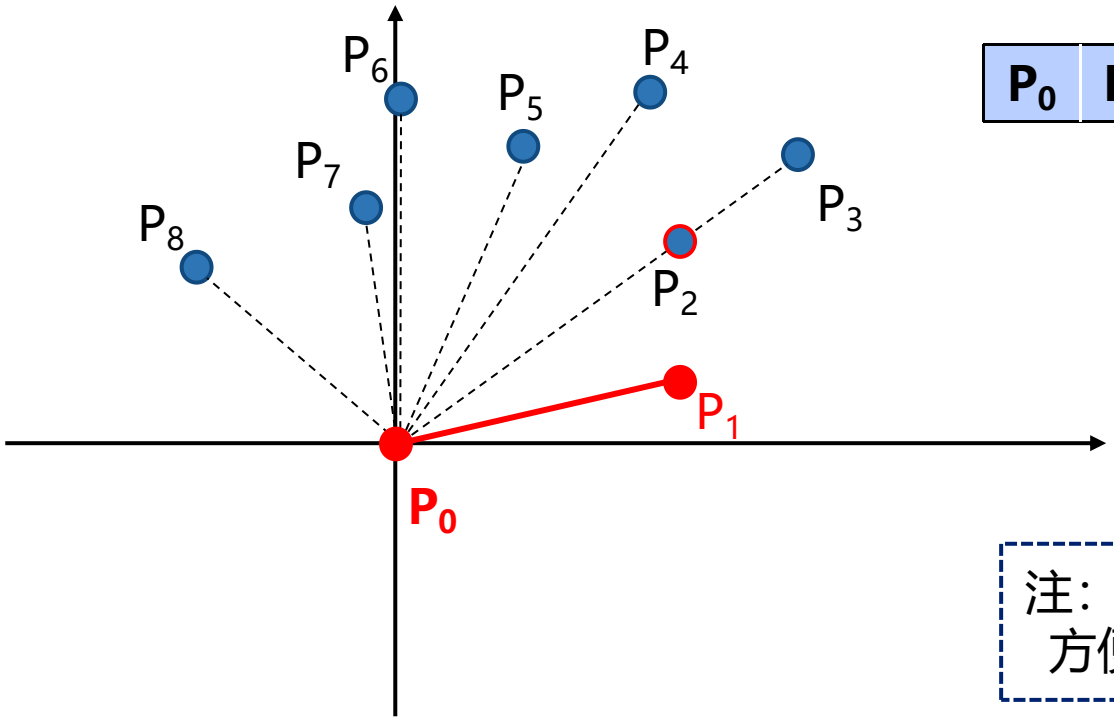
- **第二步**：计算其他各个点相对于P_0的幅角α，按α从小到大对各个点排序。当α相同时(如P_2、P_3)，距离P_0更近的点(P_2)排在前面。



由几何知识可知，P_1和P_8一定在凸包上

平面点集凸包问题：Graham扫描法

- **第三步：**目前已知P_0和P_1是凸包上的点，把它们放入栈中，用直线相连。然后按顺序找到P_1后下一个点(P_2)，观察它在前两个点连线左边还是右边。



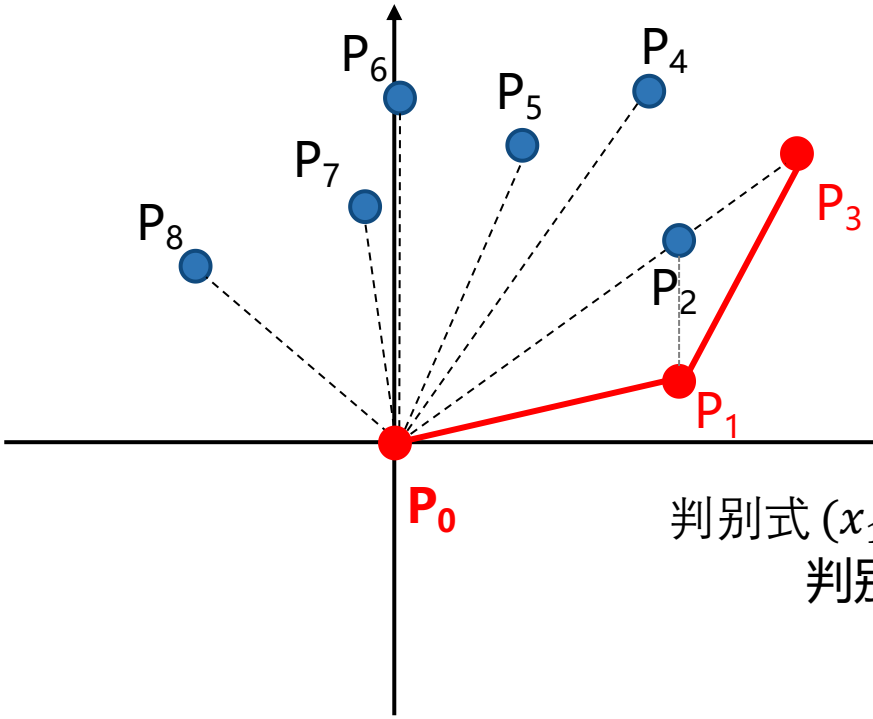
栈

P_0	P_1						

注：P_8也在凸包上，但为了方便理解，最后再连接它

平面点集凸包问题：Graham扫描法

- **第四步：**若P_2在直线左边，将其入栈，并用直线连接P_1和P_2；若该点不是同一条直线上的最后一个点，则用最后一个点(P_3)替换该点。



栈

P_0	P_1	P_3					

如何判断点在直线的左右哪边？

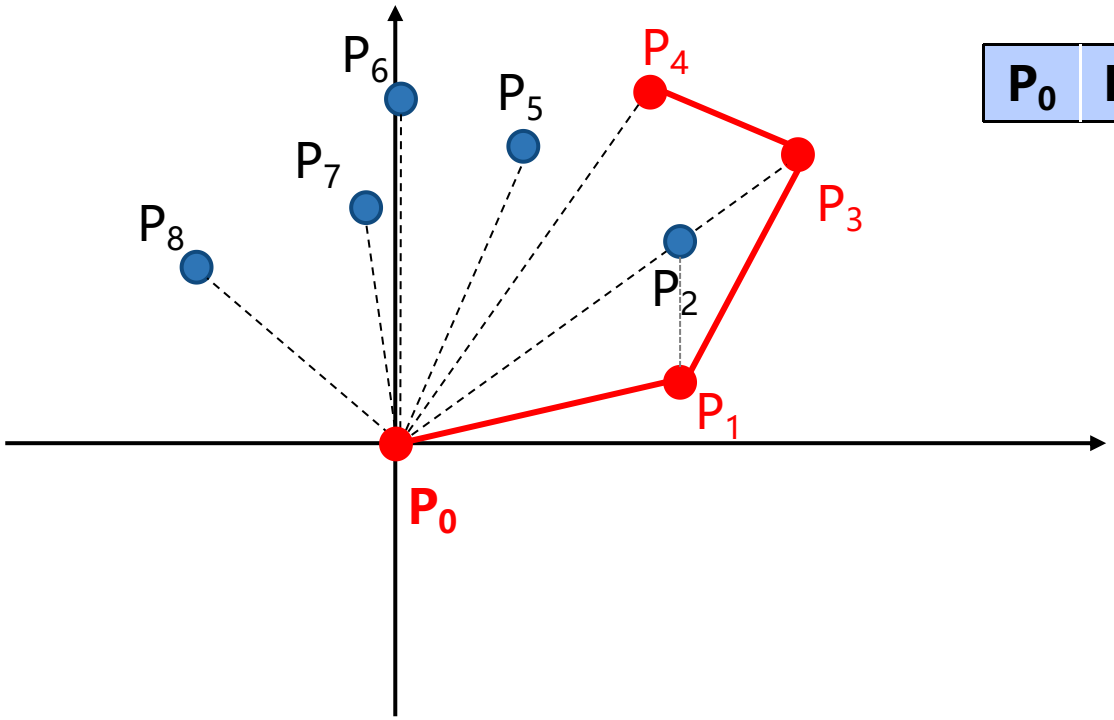
$P_0(x_1, y_1), P_1(x_2, y_2), P_2(x_3, y_3)$

判别式 $(x_1y_2 - y_1x_2) + (x_2y_3 - y_2x_3) + (x_3y_1 - y_3x_1)$

判别式为正，则P_2在直线$P_{0,1}$左边，为负在右边，为0则在一条直线上

平面点集凸包问题：Graham扫描法

- **第五步**：重复上述步骤(找下一个点→判断左右→在左边则入栈→连线→找下一个点...)。如下一个点P_4在直线$P_{1,3}$左边，则P_4入栈，连接P_3、P_4，找下一个点P_5。

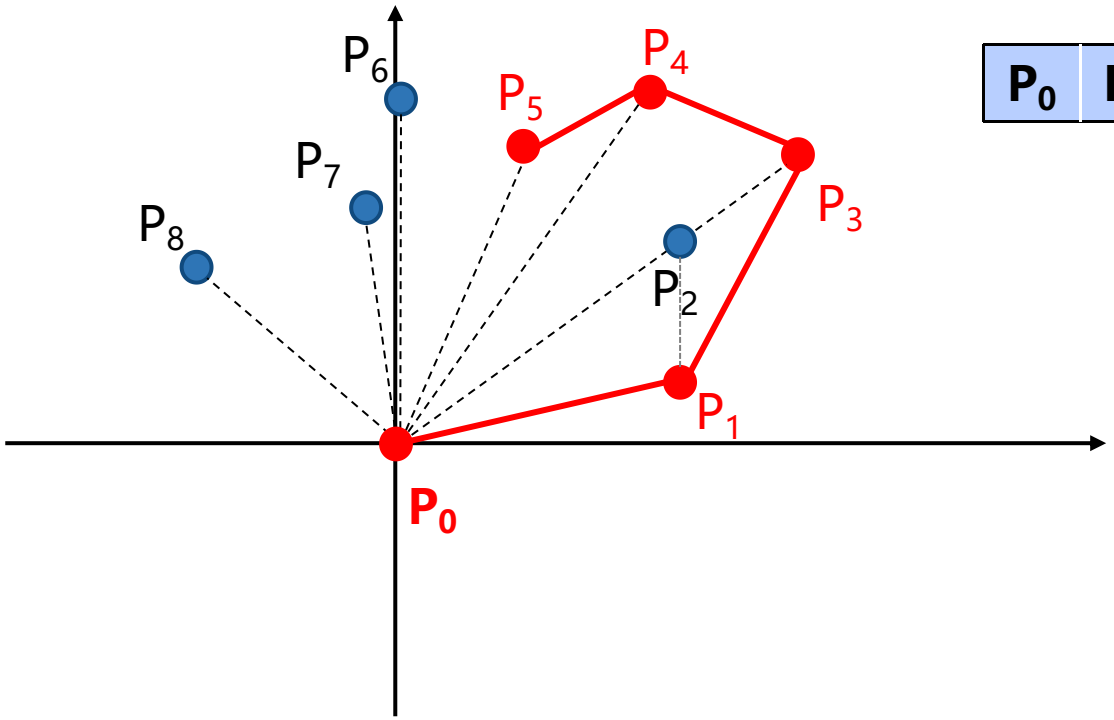


栈

P_0	P_1	P_3	P_4				

平面点集凸包问题：Graham扫描法

- **第五步：** P_5在前两个点连线$P_{3,4}$左边， P_5入栈，连线P_4和P_5，找下一个点P_6。

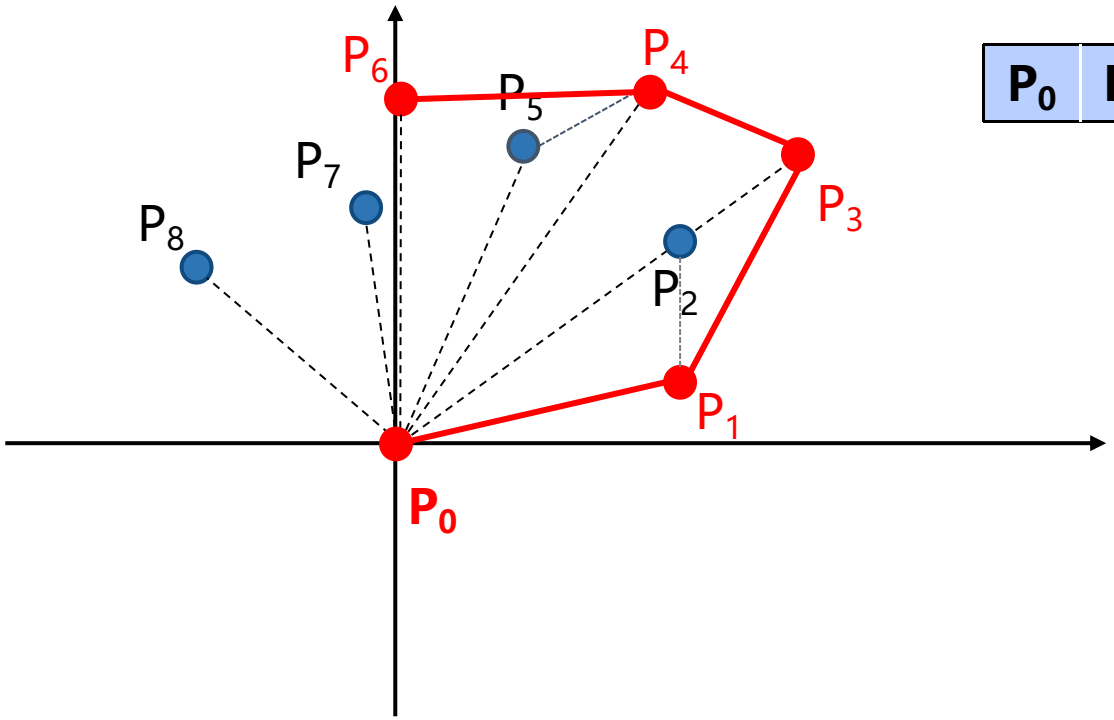


栈

P_0	P_1	P_3	P_4	P_5			

平面点集凸包问题：Graham扫描法

- **第六步：** P_6在前两个点连线$P_{4,5}$**右边**，则P_5出栈，换成P_6入栈，连线P_4和P_6，找下一个点P_7。

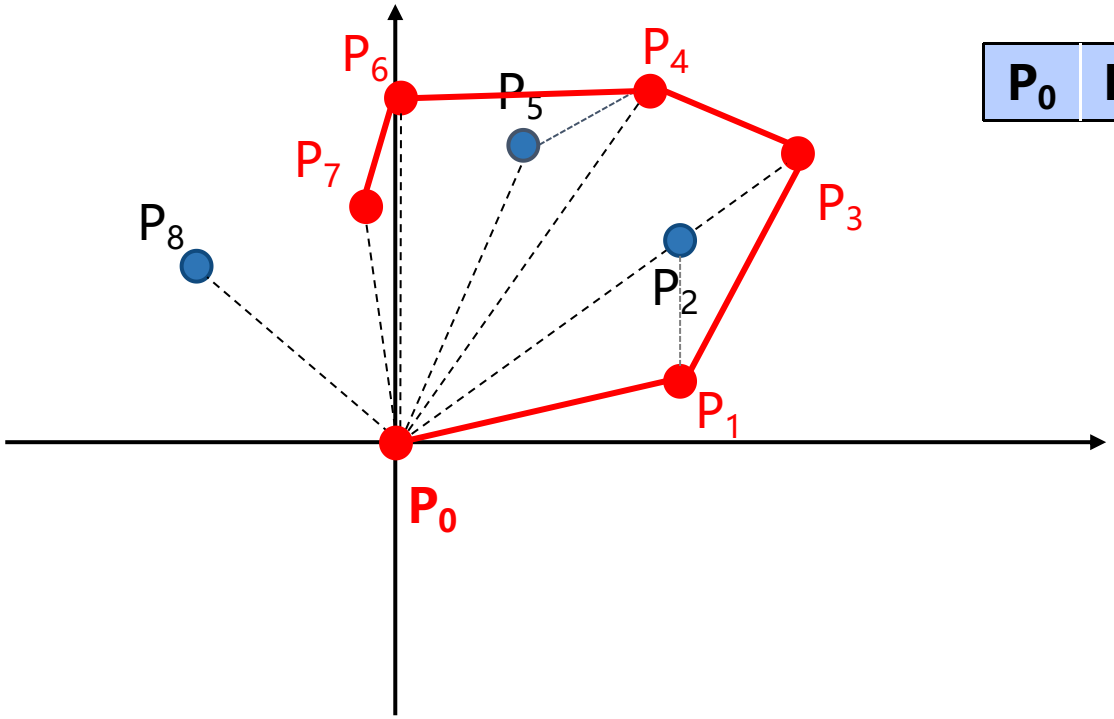


栈

P_0	P_1	P_3	P_4	P_6			

平面点集凸包问题：Graham扫描法

- **第六步：** P_7在直线$P_{4,6}$**左边**，则P_7入栈，连线P_6和P_7，找下一个点P_8。

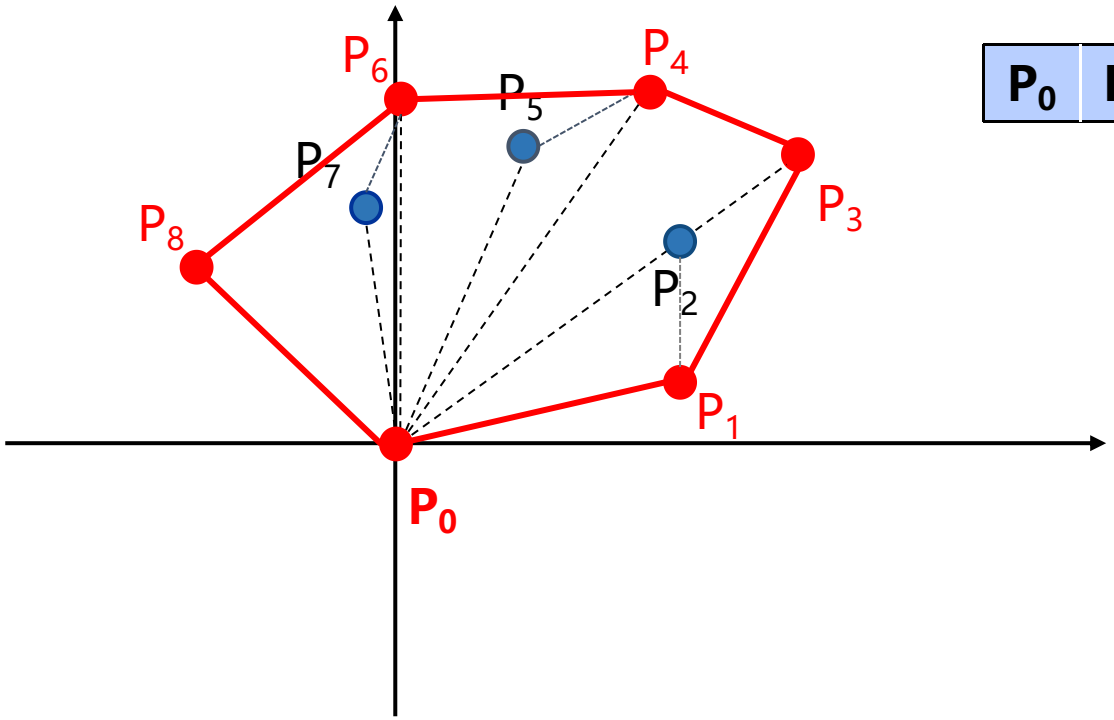


栈

P_0	P_1	P_3	P_4	P_6	P_7		

平面点集凸包问题：Graham扫描法

- **第六步：** P_8在直线$P_{6,7}$**右边**，则P_7入出栈， P_8入栈，连线P_6和P_8。至此，所有点扫描完毕，栈中元素即为凸包上的点。

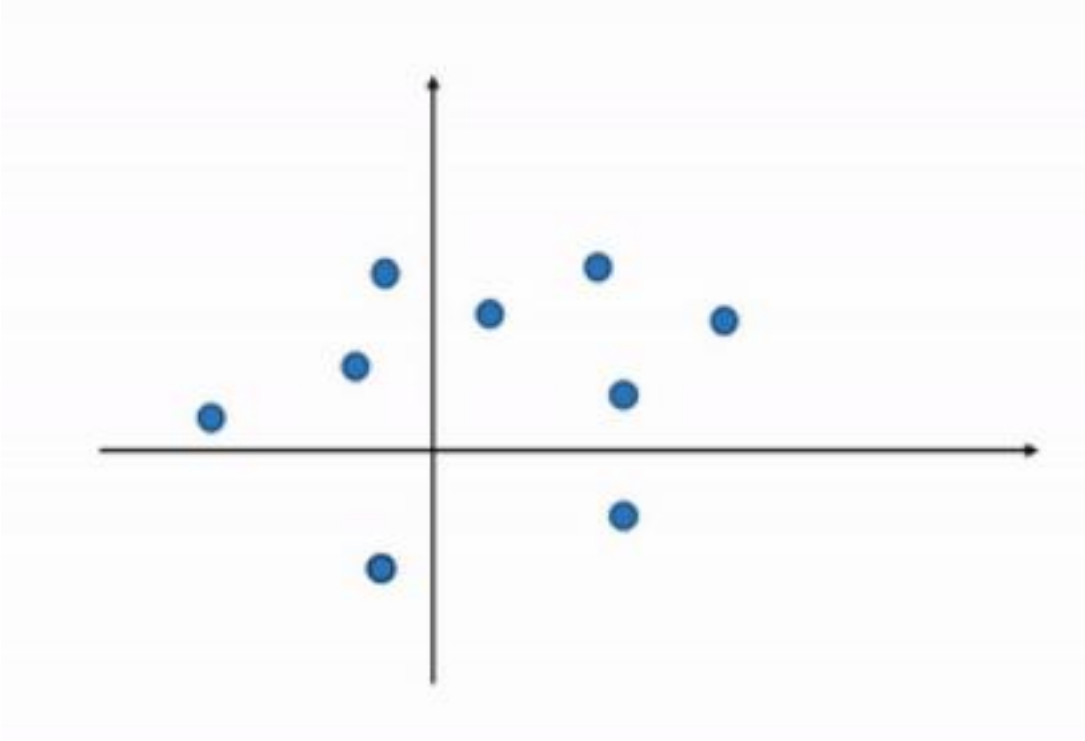


栈

P_0	P_1	P_3	P_4	P_6	P_8

平面点集凸包问题：Graham扫描法

- **时间复杂度**：开始按角度α排序过程为O(nlogn)，后面扫描过程为O(n)，总复杂度为O(nlogn)。



本章小结

（PPT004-063）分治算法概述

- 引例：伪币问题
- 基本思想

设计方法与实例

- 金块问题
- 矩阵相乘
- 大整数乘法
- 二分搜索

（PPT064-164）分治算法的应用实例

棋盘覆盖

循环赛日程表

- 归并排序（逆序对的个数）

快速排序

寻找第k小

- 平面点对
- 平面点集凸包问题

考察重点 **熟练掌握技巧**

作业

① 练习33，参见课本P463。

② 分析：当r=3时是否能在时间复杂度$O(n)$内求解选择问题？

计算：当r=7时选择算法的时间复杂度是多少？

③ 证明：由普通2×2分块矩阵乘法推导得的分治法算法，其时间复杂度为$\Theta(n^3)$，参见课本P435例14-3。

④ 推导课本P442递归式(14.7)。



第五章：分治法

算法设计与分析

主讲教师：胡一涛